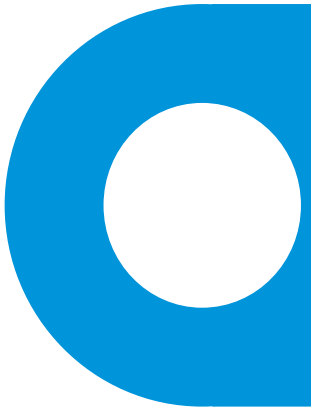


"Ss. Cyril and Methodius" University in Skopje
FACULTY OF COMPUTER
SCIENCE AND ENGINEERING



FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Introduction to JAVA

Advanced Programming

FCSE 2026

Java Programming Language – The Beginnings

- JAVA was created by Sun Microsystems in a team led by James Gosling (1991)
- Originally designed for programming home appliances (instruments)
- Difficult task because the devices are controlled by a number of different processors (controllers)
- The team developed a two-step translation method to make it easier to write different compilers for each different type of apparatus

Motivation behind the creation of the Java programming language

1. Platform Independence
2. The Need for a Language for Embedded Systems
3. Simplifying Programming
4. Security Concerns in Networked Systems
5. Growth of the Internet and Web Applications
6. Object-Oriented Programming (OOP)

The Importance of the Java Translation Process

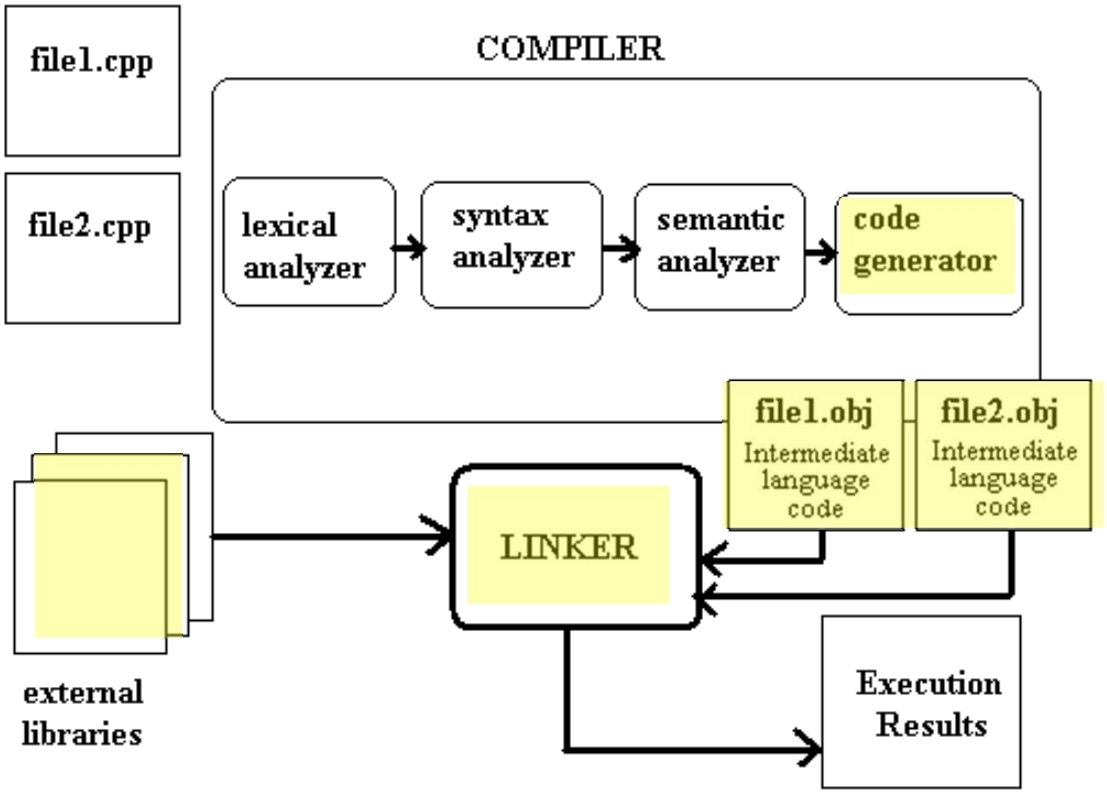
- Writing a compiler (a program to translate source code into machine code) for each type of processor would be very expensive
- Instead, an intermediate language has been developed that is the same for all types of processors: JAVA byte-code
- As a result, only a small simple program is required to translate byte-code into machine code for each type of processor before its execution

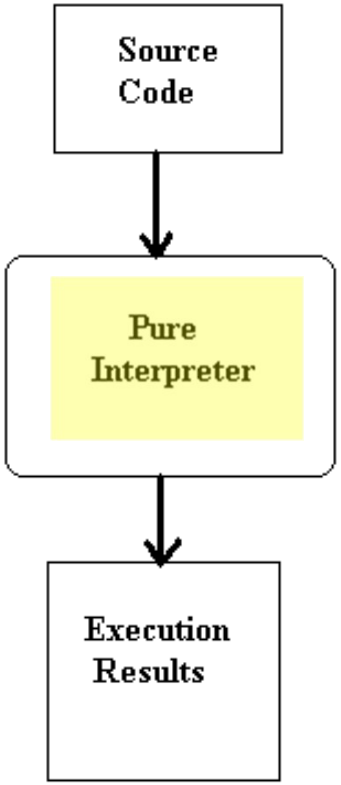
Code Generation



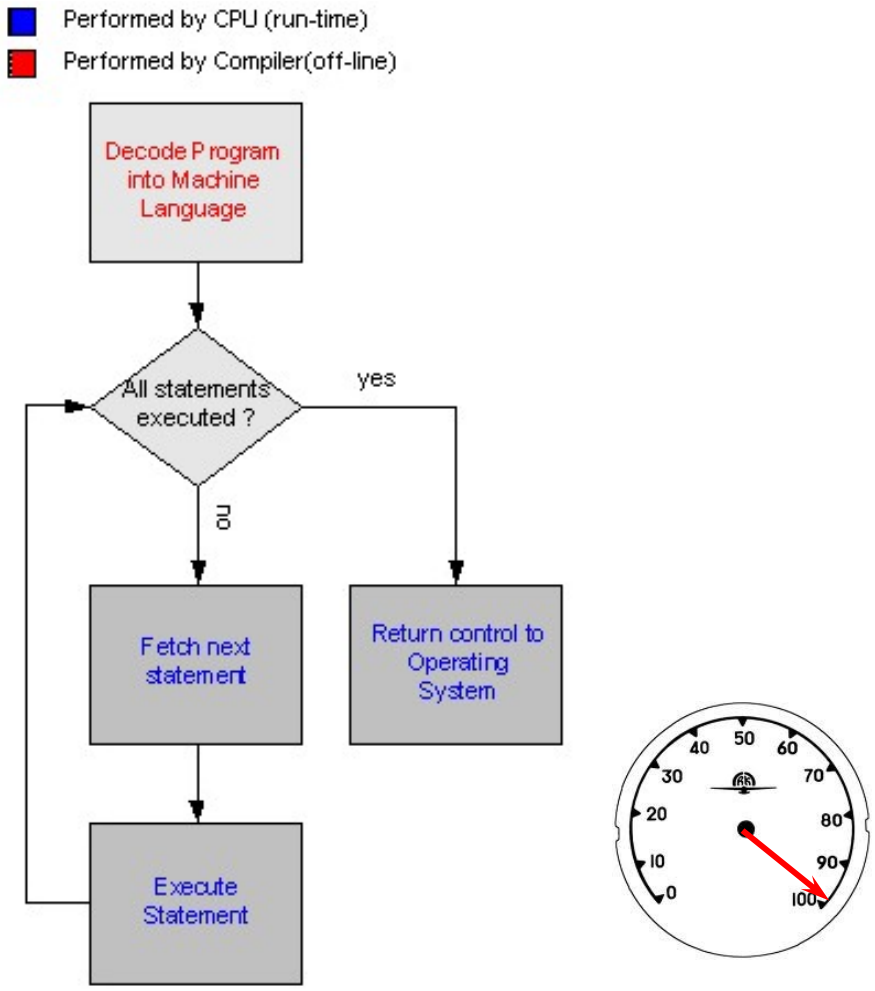
- Code Generation
  - translators (C, C++, Pascal, Fortran, ...)
  - interpreters (Basic, Javascript, Python, ...)
  - in 2 steps
    1. translation to intermediate code
    2. interpretation or just-in-time translation of intermediate code into machine code (JAVA, CLR)
  - compiler-compiler (source code generators, preprocessors)
  - yacc, bison, ...
  - Application generators (dedicated - visual) (JAG, Clarion)

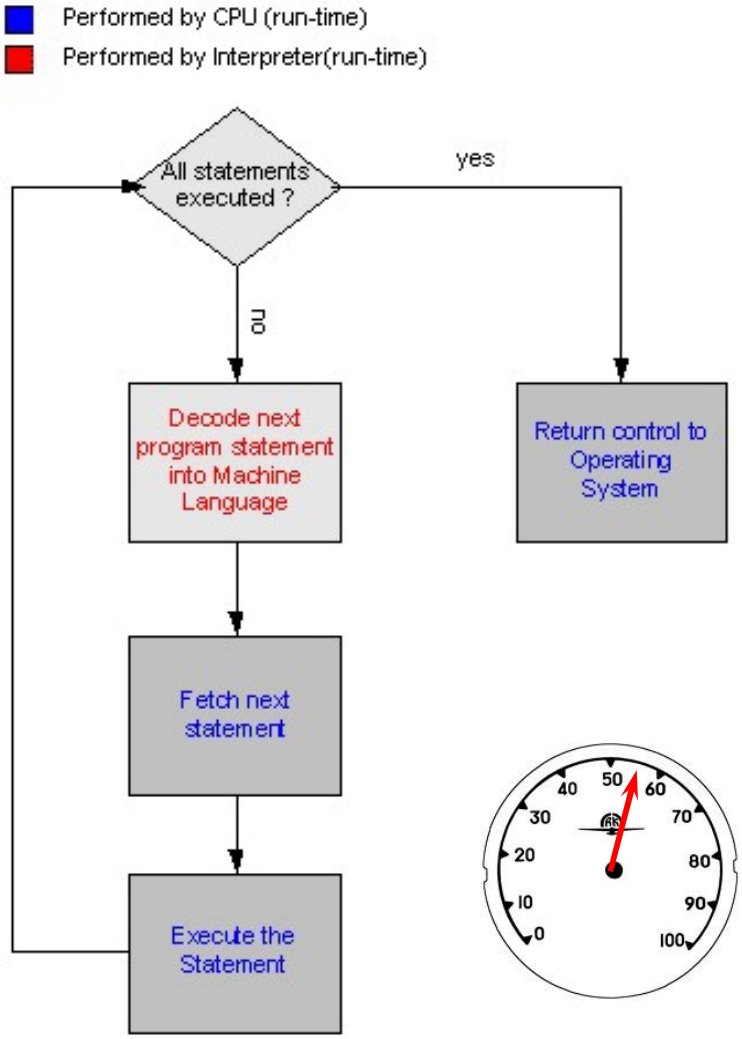
Compiler vs. Interpreter



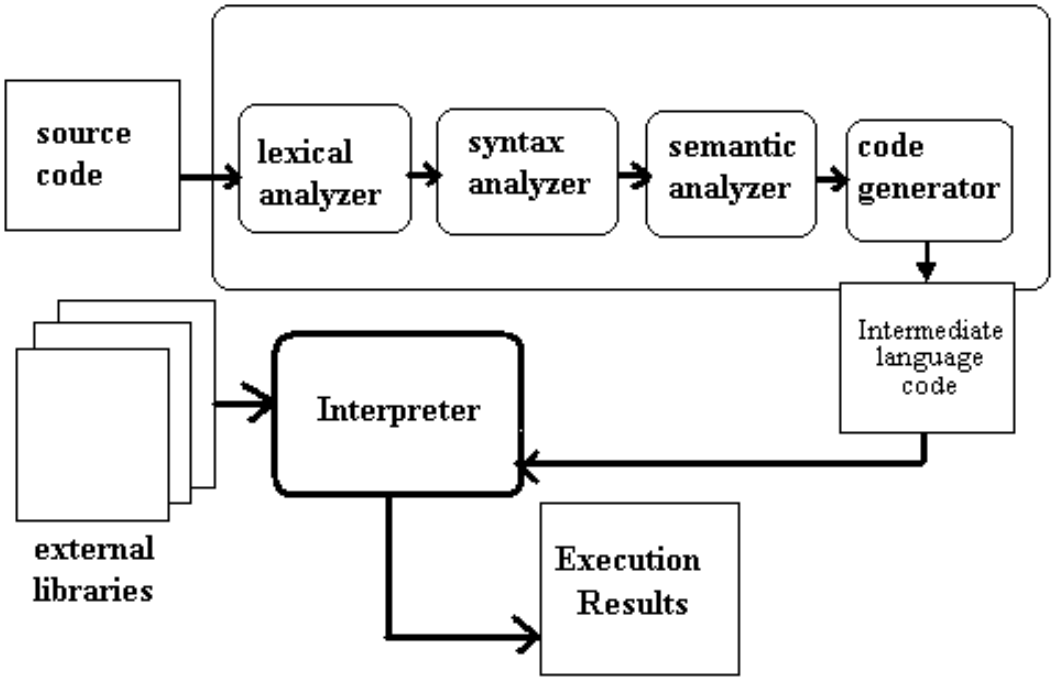


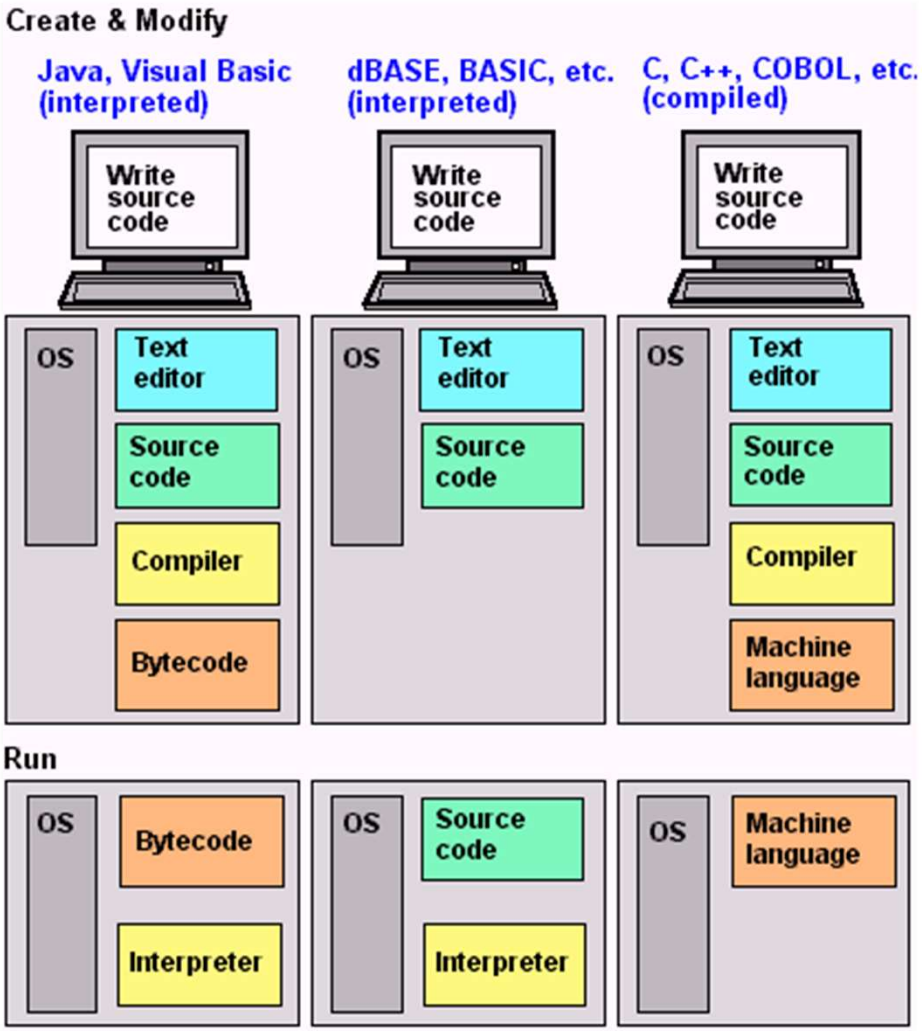
Interpreting vs. executing compiled programs





Hybrid compiler-interpreter





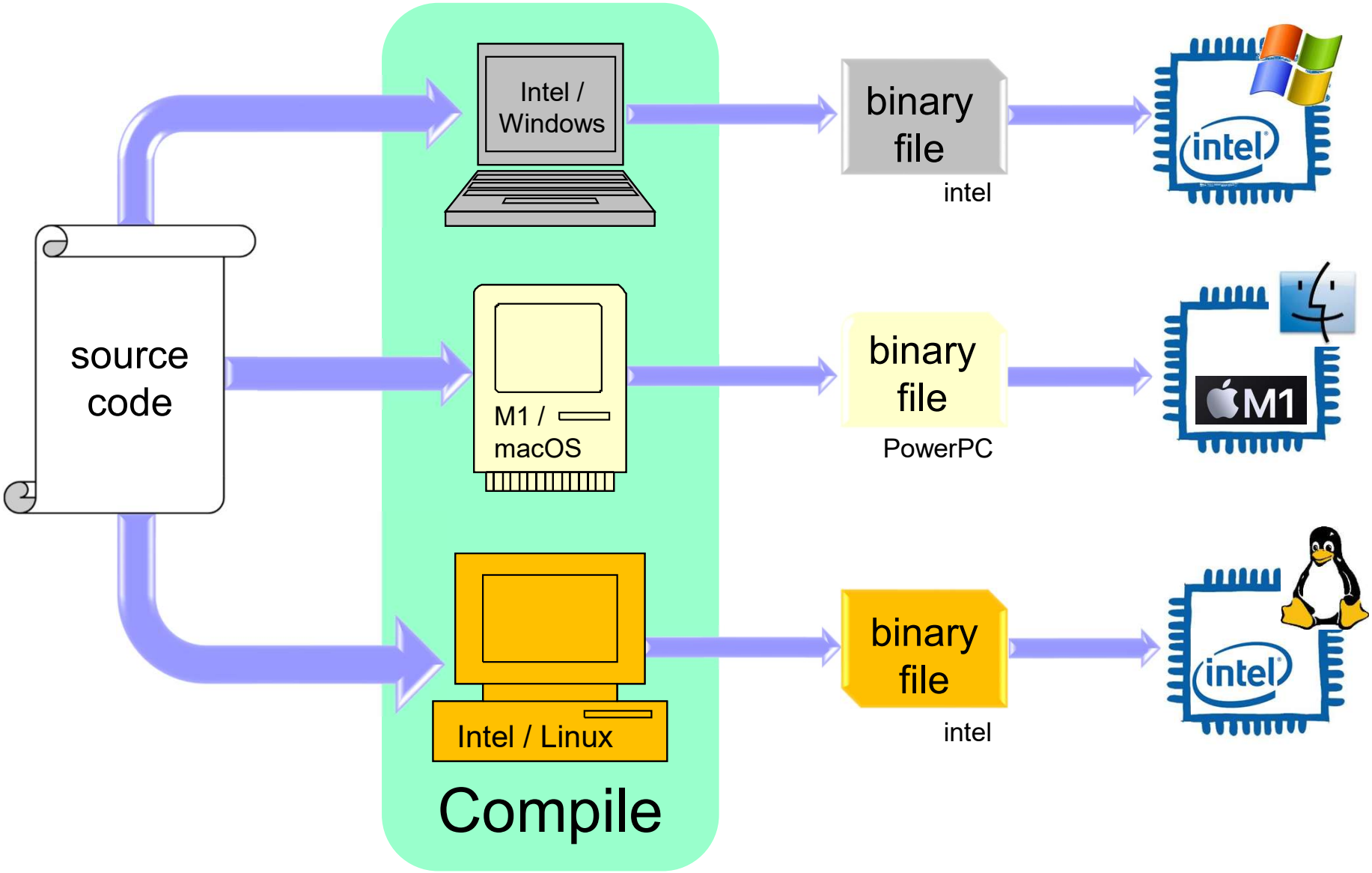
Program Development

- In many traditional execution environments, programmers were responsible for many low-level things, such as manually handling memory, ensuring code could run on different hardware, implementing security, managing errors, …
- The extent to which these services are supported on different platforms can vary significantly

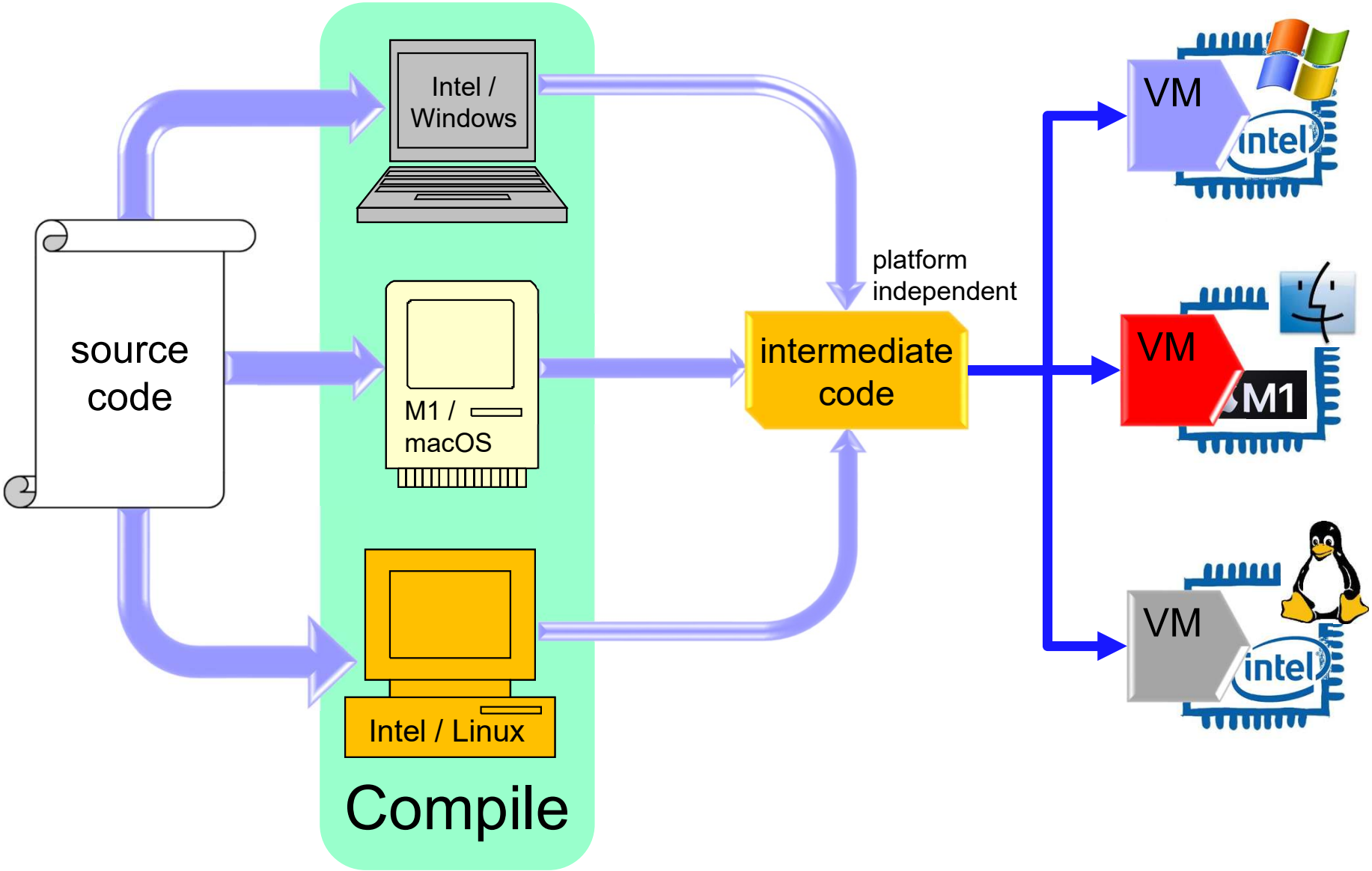
One way to overcome this

- Intermediate code / Virtual machine approach
- Virtual machine = completely isolated guest operating system installation within a normal host operating system
  - System (hardware) virtual machine (VMware, VirtualBox, Parallels Desktop, …)
  - Operating system level virtualization (Microsoft Hyper-V, Linux-Vserver, …)
  - Process (application) virtual machine (JVM, CLI, Tamarin, SpiderMonkey, (JavaScript), DOSBox, …)
- Managed / Unmanaged Code

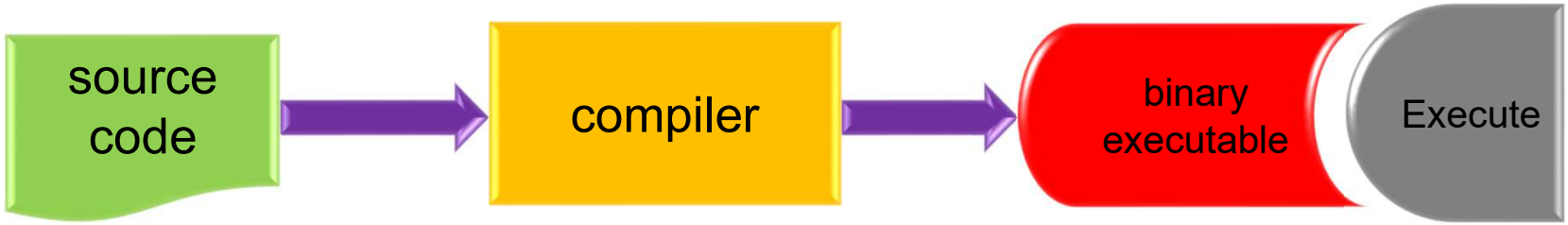
Unmanaged code



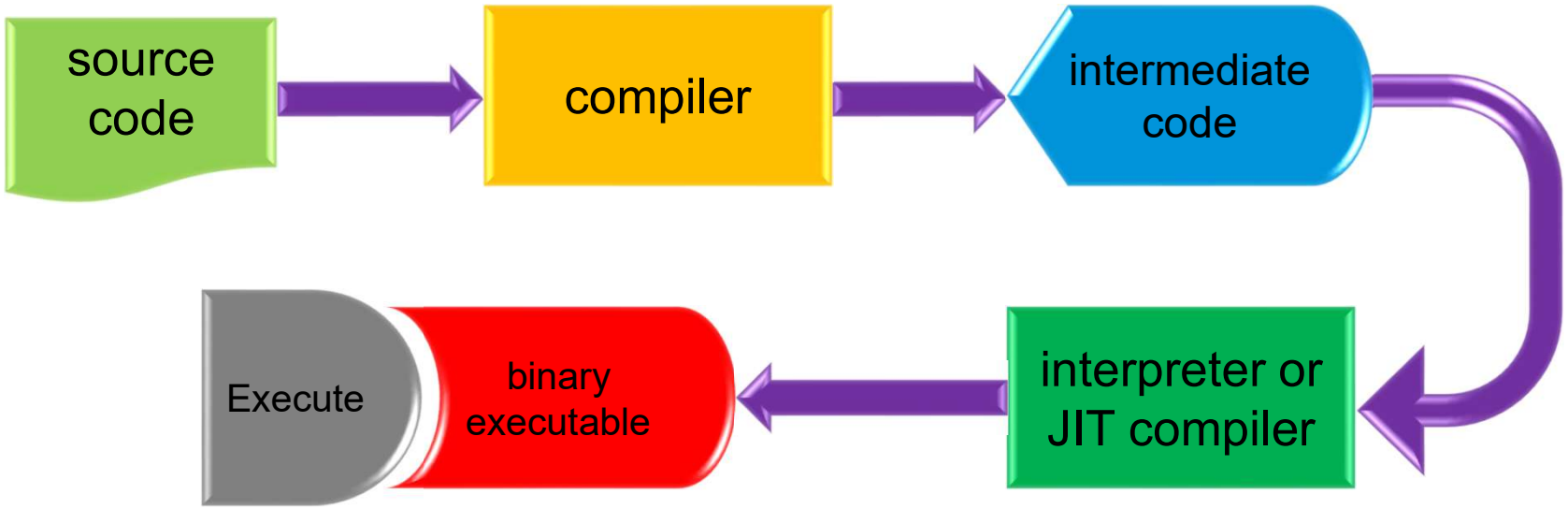
Managed code



Managed / Unmanaged code



Unmanaged code



Managed code

Byte-code and JAVA virtual machine

- Compilers in most programming languages translate a high-level programming language directly to machine code
  - Since different computers can have a different machine language, it means a separate compiler is needed for each of them
- In contrast, the Java compiler translates Java programs into byte-code, which is a "machine" language for the artificial computer called the Java Virtual Machine
  - As a byte-code, Java programs can be run on any computer (supporting the JVM), making them easily portable

Byte-code and JAVA virtual machine

- Interpreter: A program that translates into machine code (for the current computer) a program in Java byte-code at the time of its execution
- The interpreter translates and immediately executes each byte-code instruction, one after the other
- Comparing to the traditional compilation process, translating the byte-code is a relatively simple (and fast)

Terminology



- Code: Program or part of a program
- Source code (or source program): A program written in a high-level language such as Java
- It is input to the compiler program
- Object Code: The Low-Level Translated Program
- Compiler output, e.g., Java byte-code
- In the case of Java byte-code, the input to the Java byte-code interpreter



Class loader

- Java programs are divided into smaller sections called classes
- Each class is typically written in a separate file and compiled separately
- Class Loader: A program that binds byte-code to classes needed to execute a Java program
  - In compiled programming languages, this program is called linker

Syntax and semantics

- *Syntax: The grammatical rules of the language, the organization and order of words allowed in the language*
- *Semantics: The meaning of things written according to the syntactic rules of the language*

Error message

- *Bug: Program Error*
  - *The process of eliminating errors in a program is called debugging*
- *Syntax error: A grammatical error in the program*
  - *The compiler detects these errors and reports them with a corresponding message*

Error message

- *Run-time error: An error that is not detected until the program starts*
  - *The compiler cannot detect these errors*
- *Logic error: Error in the program's algorithm*
  - *The compiler cannot detect these errors, but the program does not do what it was written to do*

Objects and methods

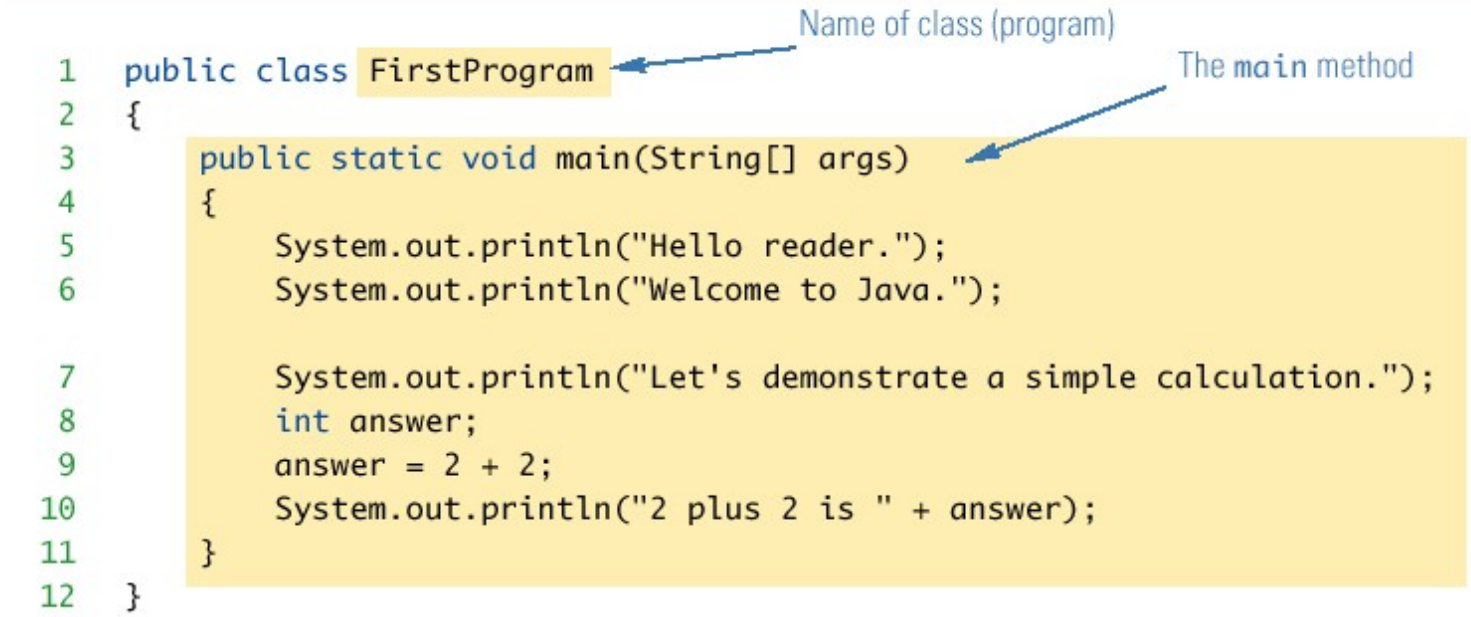
- JAVA is an object-oriented programming language
  - Programming methodology that sees programs as a composite of objects that interact with each other (through so-called methods)
  - Objects of the same kind are said to belong to the same class

Terminology comparisons

- Other high-level languages have constructs called procedures, methods, functions, and/or subroutines
  - These constructs in JAVA are called methods
  - All programming constructs in JAVA, including methods, are part of the classes

A simple JAVA application

Display 1.1 A Sample Java Program




```
1  public class FirstProgram
2  {
3      public static void main(String[] args)
4      {
5          System.out.println("Hello reader.");
6          System.out.println("Welcome to Java.");
7          System.out.println("Let's demonstrate a simple calculation.");
8          int answer;
9          answer = 2 + 2;
10         System.out.println("2 plus 2 is " + answer);
11     }
12 }
```

SAMPLE DIALOGUE 1

```
Hello reader.
Welcome to Java.
Let's demonstrate a simple calculation.
2 plus 2 is 4
```

System.out.println

- JAVA programs operate through so-called objects that perform certain actions
  - `System.out`: An object used to send information (data) for display on the screen
- The actions that objects perform are called methods
  - `println`: A method or action that the System.out object performs

JAVA/C++ similarities/differences

- The low-level syntax is virtually identical to that in C and C++
- Similar primitive data types, assignment and casting (but no pointers, no references, no unsigned types)
- Basic control structures have the same syntax and work identically (if, while, for, switch, ...)
- In Java, there is no preprocessing (nor header files) (there are packages and imports)
- There is no separation of class definitions from class implementations.
- JAVA does not support operator overloading
- JAVA does not support multiple inheritance (but does allow a class to implement multiple interfaces)
- JAVA does not provide user control for pass-by-value or pass-by-reference method arguments (there is predefined behavior)
- There is no const in JAVA
- Java has built-in protection against memory referencing out of range of a given array
- JAVA has built-in system memory management. Objects that are not in use are released automatically (there is new but no delete)
- In Java, there is increased security in the transfer of bytecode to the client and checks at runtime inside the Java Virtual Machine

Declaring Variables

C#:

```
int x;
int x=3;
```

Visual C++:

```
int x;
int x=3;
```

Java:

```
int x;
int x=3;
```

Assignments

C#:

```
xValue=7;
```

Visual C++:

```
xValue=7;
```

Java:

```
xValue=7;
```

FOR Loops

C#:

```
for (int i = 1; i <= 10; i++)
    Console.WriteLine("The number is {0}", i);
```

Visual C++:

```
for (int i = 1; i < 11; i++)
    printf("%d\n", i);
```

Java:

```
for (int i = 1; i < 11; i++)
    System.out.println("The number is " + i);
```

WHILE Loops

C#:

```
while (x < 100);
    x++;
```

Visual C++:

```
while (x < 100);
    x++;
```

Java:

```
while (x < 100);
    x++;
```

If-Else Statements

C#:

```
if (nCount < nMax)
{
  nTotal += nCount;
  nCount++;
}
else {
  nTotal += nCount;
  nCount —;
}
```

Visual C++:

```
if (nCount < nMax) {
  nTotal += nCount;
  nCount++;
  }
else {
  nTotal += nCount;
  nCount —;
  };
```

Java:

```
if (nCount < nMax) {
  nTotal += nCount;
  nCount++;
}
else {
  nTotal += nCount;
  nCount —;
}
```

Cycles

- `while (…)`
- `do … while(…)`
- `for(…; …; …)`
- `break` and `continue` in the body of cycles – just like in C and C++
- `+ break label;` (With restrictions)
- `for(type id : col)`
  - Iterating through all elements of a field or collection of objects

Operator Precedence Table in Java

Operators	Associativity	Type
++ --	Right to left	Unary postfix
++ -- + - ~ ! (type)	Right to left	Unary prefix
* / %	Left to right	Multiplicative
+ -	Left to right	Additive
<< >> >>>	Left to right	Shift
< <= > >=	Left to right	Relational
== !=	Left to right	Equality
&	Left to right	Boolean Logical AND
^	Left to right	Boolean Logical Exclusive OR
|	Left to right	Boolean Logical Inclusive OR
&&	Left to right	Conditional AND
||	Left to right	Conditional OR
?:	Right to left	Conditional
= += -= *= /= %=	Right to left	Assignment

Program Documentation

- Java comes with a program called `javadoc` that automatically extracts documentation written in a block comment
- Block comment has an extra star (`/**`)
- A well-written program is documented and clearly structured

Program Documentation

Display 1.8 Comments and a Named Constant

```
1  /**
2   Program to show interest on a sample account balance.
3   Author: Jane Q. Programmer.
4   E-mail Address: janeq@somemachine.etc.etc.
5   Last Changed: September 21, 2004.
6  */
7  public class ShowInterest
8  {
9      public static final double INTEREST_RATE = 2.5;

10     public static void main(String[] args)
11     {
12         double balance = 100;
13         double interest; //as a percent

14         interest = balance * (INTEREST_RATE/100.0);
15         System.out.println("On a balance of $" + balance);
16         System.out.println("you will earn interest of $"
17                                                 + interest);
18         System.out.println("All in just one short year.");
19     }
20
21 }
```

Although it would not be as clear, it is legal to place the definition of INTEREST_RATE here instead.

```
/**
 * @author  First Last <address@example.com>
 * @version 1.6  (current version)
 * @since   1.2  (version first added)
 */
```

Demo: IdeaProjects\SimpleJava1

SAMPLE DIALOGUE

```
On a balance of $100.0
you will earn interest of $2.5
All in just one short year.
```

Identifiers

- *Identifier:* The name of a variable, class, method, object, etc., defined in the program
  - A Java identifier cannot begin with a number, and the remaining characters must be letters, numbers, or the underscore symbol
  - Java identifiers can be of any length
  - Java is strictly **case-sensitive**.
  - *Example:* Rate, rate, and RATE are treated as three completely different identifiers.

Naming convention



- Variable names, methods, and objects are conventionally recommended to begin with lowercase letters; If the name is composed of more than one word, it is recommended that each subsequent word (after the first one) begin with a capital letter (camelCasing)

```
topSpeed    bankRate1    timeOfArrival
```

- The same is true for class names, with the difference that class names begin with a capital letter (PascalCasing)

```
FirstProgram    MyClass    String
```

Primitive types

Display 1.2 Primitive Types

TYPE NAME	KIND OF VALUE	MEMORY USED	SIZE RANGE
`boolean`	`true` or `false`	1 byte	not applicable
`char`	single character (Unicode)	2 bytes	all Unicode characters
`byte`	integer	1 byte	−128 to 127
`short`	integer	2 bytes	−32768 to 32767
`int`	integer	4 bytes	−2147483648 to 2147483647
`long`	integer	8 bytes	−9223372036854775808 to 9223372036854775807
`float`	floating-point number	4 bytes	$-3.40282347 \times 10^{+38}$ to $-1.40239846 \times 10^{-45}$
`double`	floating-point number	8 bytes	$\pm 1.76769313486231570 \times 10^{+308}$ to $\pm 4.94065645841246544 \times 10^{-324}$

Assignment in Primitive Types

- In Java, an assignment command is used to change the value of variables
  - The sign (=) is used as an assignment operator
  - When the assignment operator is executed, the expression on the right side is first evaluated, and then the resulting value is assigned to the variable on the left side

```
distance = rate * time;
```

  - The assignment operation is automatically performed from right to left

```
number2 = number1 = 3;
```

Initialization of variables

- A variable that is declared, but not assigned a value, is said to be *uninitialized*
- In specific cases, predefined values are assigned to the uninitialized variable
  - It's better not to rely on this
  - Explicit initialization of variables is recommended to improve program clarity

```
int x=2, z;
z=x+23; //OK
```

```
int x=2, z;
x=z+23; //error
```

Assignment compatibility

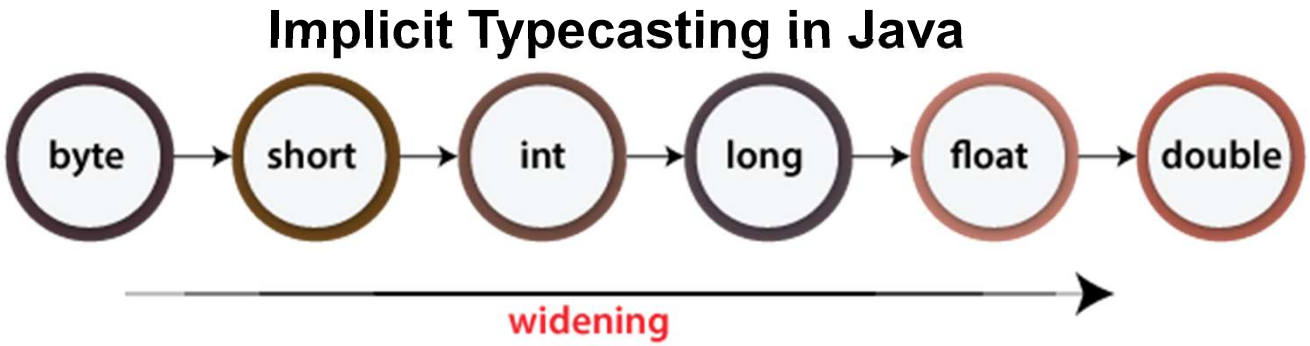
- In general, a value of any data type in the following list can be assigned to a variable of any type that appears to the right of it

  `byte→short→int→long→float→double`

  `char→int→…→double`

  `boolean`

- Explicitly casting the value of a variable is required to add it to a variable whose type appears to the left of the variable type in the previous list (e.g. `double` in `int`)
- In Java, `int` value cannot be assigned to a `boolean` variable and vice versa



Casting types

- Casting types takes a value of one type and generates a value of another type with an "equivalent" value
  - If n and m are integers and need to be divided, to obtain the true decimal result, at least one of the numbers must be cast in a floating-point type before the division operation can be performed
  - double ans = n / (double)m;

Casting types

- When casting a decimal number to an integer, the decimal part is discarded (ignored)
- (int)2.9 is evaluated at 2, not 3
- It is not allowed to initialize an int variable with a double value without explicit casting

```
int i = 5.5; // Illegal
int i = (int)5.5; // Correct
```

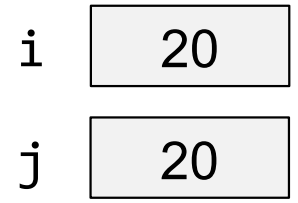
Data types in Java

- Java data types are categorized into two main groups: Primitive Data Types and Non-Primitive (Reference) Data Types.
- Basic (primitive) types are value types
  - boolean, byte, char, short, int, long, float, double
- All other types are reference types:
  - all objects, arrays (which behave like classes), wrapper classes for primitive types,
  - Example: Myclass, int [], Integer, …

Value / Reference types

```
int i, j;
i = 20;
j = i;
```

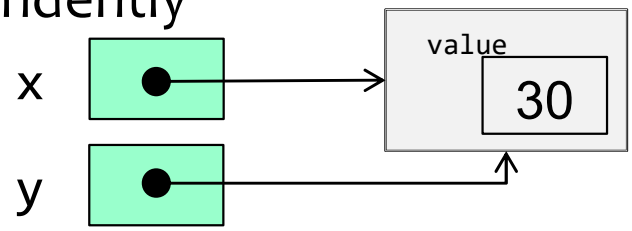
int is value type



- After this there are two (different) memory locations that store the value of 20 and each can be changed independently
- class Vector …

```
Vector x, y;
x = new Vector();
x.value = 30; // Value is a field defined in Vector class
y = x;
System.out.println(y.value);
y.value = 50;
System.out.println(x.value);
```

Vector is reference type




```
30

50
```

```
y = null;
```

- There is only one object of the Vector type, and x and y both refer to it

Value / Reference types example

```
public class Main {
        public static void main(String[] args) {
            int i,j;
            i=20;
            j=i;
            System.out.println("i = " + i + ", j = " + j);
            i=30;
            System.out.println("i = " + i + ", j = " + j);

            Vector x,y;
            x=new Vector();
            x.value=30;
            y=x;
            System.out.println("x = " + x.value + ", y = " + y.value);
            y.value=35;
            System.out.println("x = " + x.value + ", y = " + y.value);
```

```
i = 20, j = 20
i = 30, j = 20
x = 30, y = 30
x = 35, y = 35
i = 30, j = 20
x = 50, y = 50
```

Value / Reference types example

```
        ChangeValue(j,50);
        ChangeValue(y,50);
        System.out.println("i = " + i + ", j = " + j);
        System.out.println("x = " + x.value + ", y = " + y.value);
    }

    public static void  ChangeValue(int x, int newVal) {
        x = newVal;
    }

    public static void ChangeValue(Vector x, int newVal) {
        x.value = newVal;
    }
}
```

```
i = 20, j = 20
i = 30, j = 20
x = 30, y = 30
x = 35, y = 35
i = 30, j = 20
x = 50, y = 50
```

Distinction between Class and Primitive Parameter Types

- A method cannot change the value of a variable of a primitive data type if it is an argument to that method
- Conversely, a method can change the value of a variable of a class-type instance if it is an argument to that method

Classes

- Classes are a very important feature of a programming language to enable object-oriented programming
- Java programming consists of defining classes
- Every program is a class
- Each auxiliary software consists of classes
- All types defined by the programmer are classes
- Classes are central to Java

Class is type

- A class is a special kind of user-defined type that can be used from which variables can be instantiated
- The value of a class type is the object or instance of the class
- If A is a class, then the phrases "bla is of type A," "bla is an object of class A," and "bla is an instance of class A" have the same meaning
- The class defines the type of data that the object contains, as well as the actions it can take

Values of primitive types vs. values of class types

- A primitive type value is a single data
- The value of a class type or object consists of multiple data as well as actions called methods
- All objects of the class have the same methods
- All objects in the class have the same data (i.e., names, types, and data numbers)
- For a given object, each data can have a different value

Class Definition

- The definition of a class specifies the data members and methods that each of its objects will have
- These data elements and methods are usually referred to as object members
- Data elements are called instance variables

Classes

- In contrast to C++, access modifiers in JAVA are specified before each class element (data/method)
- Java provides four levels of access control
  - Default : When no access modifier is specified
  - `public`
  - `private`
  - `protected`

`new` operator

- An object is named or declared as a variable of the class:

```
ClassName  classVar;
```

- The **`new`** operator is used to create an object of the class and associate it with the name of the variable:

```
classVar = new ClassName();
```

- The same can be written as follows:

```
ClassName classVar = new ClassName();
```

Variables and Instance Methods

- Instance variables can be defined as in the following two examples:

```
public String  instanceVar1;
public int  instanceVar2;
```

- Access to instance variables is done as follows:

```
objectName.instanceVar1
objectName.instanceVar2
```

- Access to methods is the same as to instance variables

```
classVar.myMethod();
```

Importing packages and classes

- Libraries in Java are called *packages*
  - A package is a collection of classes that are grouped together to be easily accessible to any program
  - To use a class belonging to a particular packet, it is necessary to introduce it into the program with the ***import*** command
  - Classes from the **`java.lang`** package are imported automatically into every Java program

```
import java.text.NumberFormat;
 // import theNumberFormat class only
import java.text.*;
 //import all the classes in package java.text
```

File and Location Names

- Java files must have the same name as the classes defined in them with the suffix .java
- Each source file should contain exactly one public class or interface.
  - You may include secondary package-private or private classes within the same file, but the public class must appear first.

enum

- In JAVA, enum is a special data type used to define a fixed set of predefined constants.
  - It acts as "class" that describes a group of constants (that are implicitly public static final)

```
enum Level {
   LOW,
   MEDIUM,
   HIGH
}
Level myVar = Level.MEDIUM;
System.out.println(myVar);
```

```
for (Level myVar : Level.values()) {
  System.out.println(myVar);
}
```

```
MEDIUM
```

More about methods

- There are two types of methods:

1. Methods that compute something and return value
2. Methods that perform an action
   - This type of method does not return value and is called void methods

- The way methods are defined and how they are used differs slightly

More about methods

- A method that returns a value must specify the type of that value in its header:

```
public typeReturned methodName(paramList)
```

- The void method uses the void keyword in the header and shows that it does not return a value:

```
public void methodName(paramList)
```

main

- A program in Java is a class that has a main method
- When a command is given to start a program written in Java, the run-time system calls the main method
- main is a void method:

```
public static void main(String[] args)
```

- In Java **there are no global variables**

Global variables

- In Java **there are no global variables**

Parameters of primitive types

- Methods can receive additional data through the list of parameters
- These parameters are called formal parameters

Parameters of primitive types

- In the following example, the value of each argument (not the name of the variable) is added to the corresponding parameter of the method
- This method of passing the values of arguments to formal parameters is called a **call-by-value** mechanism

Use of formal parameters as local variables (1 of 5)

Display 4.6 A Formal Parameter Used as a Local Variable

This is the file `Bill.java`.

```
1   import java.util.Scanner;

2   public class Bill
3   {
4       public static double RATE = 150.00; //Dollars per quarter hour

5       private int hours;
6       private int minutes;
7       private double fee;
```

(continued)

Use of formal parameters as local variables (2 of 5)

Display 4.6 A Formal Parameter Used as a Local Variable

```
 8       public void inputTimeWorked()
 9       {
10           System.out.println("Enter number of full hours worked");
11           System.out.println("followed by number of minutes:");
12           Scanner keyboard = new Scanner(System.in);
13           hours = keyboard.nextInt();
14           minutes = keyboard.nextInt();
15       }

16       public double computeFee(int hoursWorked, int minutesWorked)
17       {
18           minutesWorked = hoursWorked*60 + minutesWorked;
19           int quarterHours = minutesWorked/15; //Any remaining fraction of a
20                                                // quarter hour is not charged for.
21           return quarterHours*RATE;
22       }

23       public void updateFee()
24       {
25           fee = computeFee(hours, minutes);
26       }
```

computeFee uses the parameter minutesWorked as a local variable.

Although minutes is plugged in for minutesWorked and minutesWorked is changed, the value of minutes is not changed.

(continued)

Use of formal parameters as local variables (3 out of 5)

Display 4.6 A Formal Parameter Used as a Local Variable

```
27      public void outputBill()
28      {
29          System.out.println("Time worked: ");
30          System.out.println(hours + " hours and " + minutes + " minutes");
31          System.out.println("Rate: $" + RATE + " per quarter hour.");
32          System.out.println("Amount due: $" + fee);
33      }
34  }
```

(continued)

Use of formal parameters as local variables (4 out of 5)

Display 4.6 A Formal Parameter Used as a Local Variable

This is the file `BillingDialog.java`.

```
1  public class BillingDialog
2  {
3     public static void main(String[] args)
4     {
5          System.out.println("Welcome to the law offices of");
6          System.out.println("Dewey, Cheatham, and Howe.");
7          Bill yourBill = new Bill();
8          yourBill.inputTimeWorked();
9          yourBill.updateFee();
10         yourBill.outputBill();
11         System.out.println("We have placed a lien on your house.");
12         System.out.println("It has been our pleasure to serve you.");
13       }
14 }
```

(continued)

Use of formal parameters as local variables (5 out of 5)

Display 4.6 A Formal Parameter Used as a Local Variable

SAMPLE DIALOGUE

```
Welcome to the law offices of
Dewey, Cheatham, and Howe.
Enter number of full hours worked
followed by number of minutes:
3 48
Time worked:
2 hours and 48 minutes
Rate: $150.0 per quarter hour.
Amount due: $2250.0
We have placed a lien on your house.
It has been our pleasure to serve you.
```

The `this` parameter

- this is kind of a hidden parameter to every method call, although it does not appear in the list of parameters
- When a given method is invoked, this parameter is automatically passed when the method is called
- this is a reference variable that points directly to the current object (represents the specific instance of the class whose method or constructor is currently executing)

Information Hiding and Encapsulation

- *Information hiding* is the practice of separating how a class is used from the details of its implementation
  - *Abstraction* is another term that expresses the concept of hiding details in order to reduce information overload
- *Encapsulation* means that the data and the methods are combined into a single entity (object) that hides the implementation details
  - Knowledge of the details is unnecessary since the interaction takes place through a well-defined simple interface
  - In Java, hiding the details is performed by marking data and methods as **`private`**

API and ADT

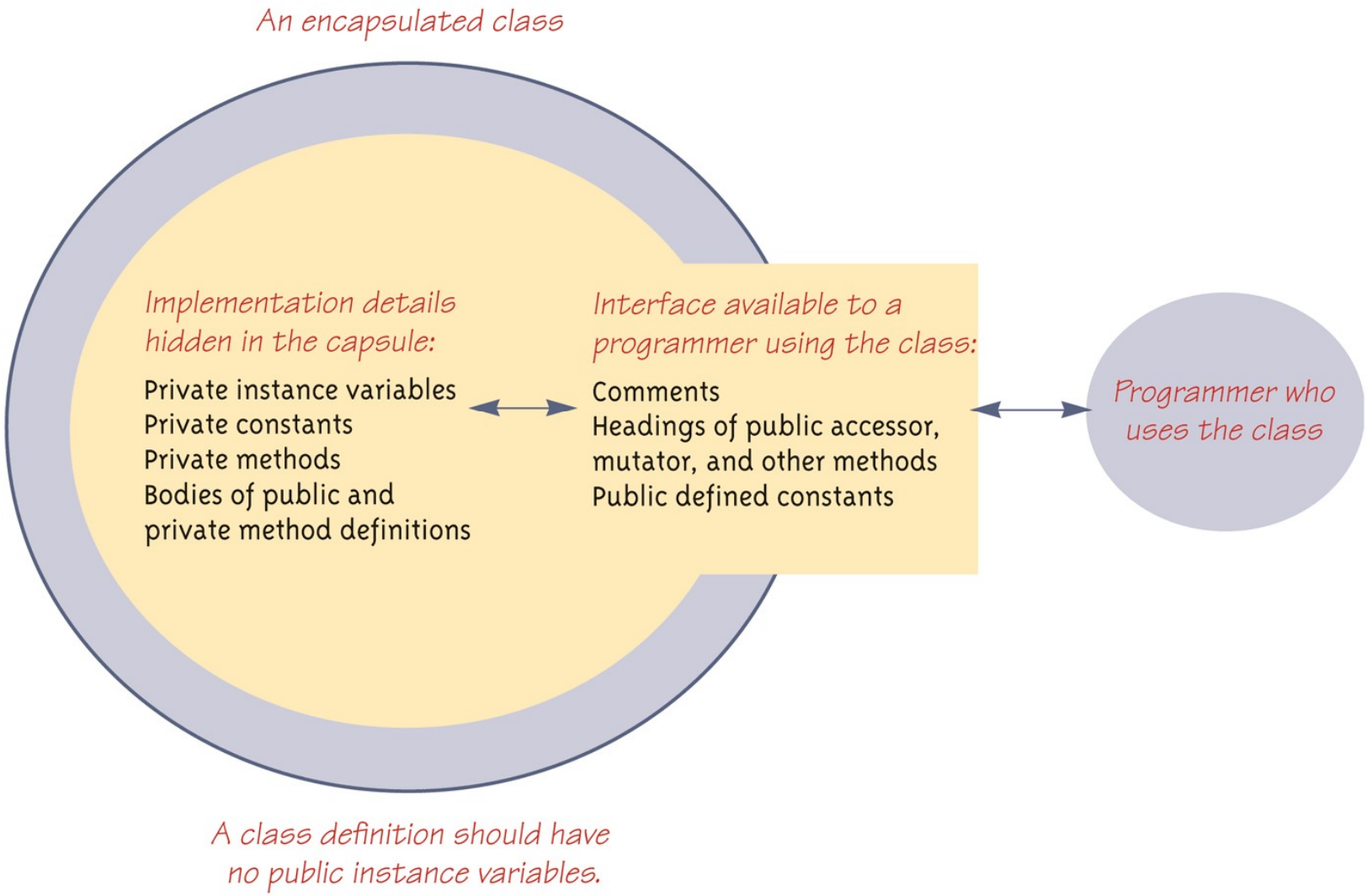
- The API or *application programming interface* for a given class is the description of the use of that class
  - A programmer needs to understand the API of a given class in order to be able to use it
- ADT or *abstract data type* is a data type that is defined using good techniques for information hiding

`public` and `private` modifiers

- The `public` modifier means that there is no limitation where instance variables or methods can be used
- The modifier `private` means that instance variables and methods cannot be accessed outside the class
- It is recommended that all variables of the instance be declared private
- Most methods are declared `public` and allow controlled access to the object
- Typically, methods are declared `private` only if they are only used as auxiliary methods within other methods in the class

Encapsulation

Display 4.10 Encapsulation



Overload

- *Overloading* is when two or more methods in a class have the same name
- To be valid, methods must have different signatures
  - The signature consists of the name of the method and the type of its parameters
  - Different method signatures must have a different number and/or type of parameters

Overloading cannot be done based on the type of data being returned

- The signature of the method includes only the name and type of parameters that the method receives
  - The signature does not include the type of data being returned
- In Java, it is not allowed to define methods that have the same names and parameters, only differing in the type of data they return

Operators can't be overloaded

- Although many programming languages, such as C++, allow operator overloading (+, -, etc.), in Java this is not allowed
- Only the syntax of ordinary methods can be used to define the desired action

Constructors

- The constructor is invoked when an object of a given class is created using the **new** operator

```
ClassName objectName = new ClassName(anyArgs);
```

  - This is the only way to validly invoke a constructor: the constructor cannot be invoked as an ordinary method
- If the class constructor is called again (using the new operator), the original object is deleted and a new object is created

Other methods can be called in the constructor

- The first action of the constructor is to create the object with the variables of the instance
- As a consequence, it is permissible to refer to other methods when defining the constructor
  - As an example, modification methods can be used to set the values of variables on an instance
  - Moreover, it is possible to refer to another constructor

The constructors have `this` parameter

- As with other conventional methods, each constructor also has the **`this`** parameter
- This parameter can be used explicitly within the constructor
- The first action of the constructor is to automatically create the object with all its variables (fields)
- This parameter then refers to the object created by the constructor

Constructor without arguments

- If you do not explicitly write a constructor for the class, Java will automatically create a predefined (default) constructor with no arguments, which does not initialize anything but allows the object to be created
- If you write at least one constructor in a class, the predefined constructor is not generated automatically
- If you write at least one constructor in your class, always define the constructor without arguments

Automatic instance variable initialization

- In Java, instance variables **are automatically initialized**
  - The `boolean` type is initialized with the false value
  - The other primitive types are initialized with the zero value for the particular data type
  - Class types are initialized with null
- However, it is good practice for all instance variables to be explicitly initialized in the constructor
- Note: This applies only to the data elements of the object, local variables are not initialized automatically

Static methods

- A *static* method is one that can be called without a calling object
- The *static* method belongs to the class and its definition is given in the definition of the class
- When a static method is defined, the **`static`** keyword appears in the method header

```
public static returnedType myMethod(parameters)
{ . . . }
```

- Static methods are invoked through classes instead of invoking objects

```
returnedValue = MyClass.myMethod(arguments);
```

Calling non-static methods from static methods

- A static method cannot access an instance variable and cannot invoke a non-static method of the class
  - The static method does not have `this`
  - Static method can call another static method

`main` in any class

- Although the main method is usually written in a separate class, it can still be part of the definition of any regular class
  - This ensures that the class can be started as a program
  - The main method included in the definition of a regular class can be especially useful when it contains a diagnostic code for the class

Class with `main` Method (1 of 4)

Display 5.3 Another Class with a `main` Added

```
1   import java.util.Scanner;

2   /**
3   Class for a temperature (expressed in degrees Celsius).
4   */
5   public class Temperature
6   {
7       private double degrees; //Celsius

8       public Temperature()
9       {
10          degrees = 0;
11      }

12      public Temperature(double initialDegrees)
13      {
14          degrees = initialDegrees;
15      }

16      public void setDegrees(double newDegrees)
17      {
18          degrees = newDegrees;
19      }
```

Note that this class has a `main` *method and both static and nonstatic methods.*

(continued)

Class with **`main`** Method (2 of 4)

Display 5.3 Another Class with a `main` Added

```
20      public double getDegrees()
21      {
22          return degrees;
23      }

24      public String toString()
25      {
26          return (degrees + " C");
27      }
28
29      public boolean equals(Temperature otherTemperature)
30      {
31          return (degrees == otherTemperature.degrees);
32      }
```

(continued)

Class with `main` Method (3 of 4)

Display 5.3 Another Class with a `main` Added

```
33      /**
34       Returns number of Celsius degrees equal to
35       degreesF Fahrenheit degrees.
36      */
37      public static double toCelsius(double degreesF)
38      {
39
40           return 5*(degreesF - 32)/9;
41      }

42      public static void main(String[] args)
43      {
44          double degreesF, degreesC;
45
46          Scanner keyboard = new Scanner(System.in);
47          System.out.println("Enter degrees Fahrenheit:");
48          degreesF = keyboard.nextDouble();
49
50          degreesC = toCelsius(degreesF);
51
```

Because this is in the definition of the class `Temperature`*, this is equivalent to* `Temperature.toCelsius(degreesF)`.

(continued)

Class with `main` Method (4 of 4)

Display 5.3 Another Class with a `main` Added

```
52          Temperature temperatureObject = new Temperature(degreesC);
53          System.out.println("Equivalent Celsius temperature is "
54                                  + temperatureObject.toString());
55      }
56  }
```

Because `main` *is a static method,* `toString` *must have a specified calling object like* `temperatureObject`.

SAMPLE DIALOGUE

```
Enter degrees Fahrenheit:
212
Equivalent Celsius temperature is 100.0 C
```

class WorkoutDuration

```
import java.util.Objects;

public class WorkoutDuration {
    protected int hours;
    protected int minutes;

    WorkoutDuration() {
        hours=minutes=0;
    }
    WorkoutDuration(int m) {
        this(m/60,m%60);
    }
    WorkoutDuration(int h, int m) {
        hours=h+m/60;
        minutes=m%60;
    }
    public void addHours(double h) {
        addMinutes(h*60);
    }
    public void addMinutes(double m) {
        minutes+=(int)m;
        hours+=minutes/60;
        minutes%=60;
    }
```

class WorkoutDuration

```
public double getHours() {
    return hoursMinutesToDecHours(hours,minutes);
}
public int getMinutes() {
    return hoursMinutesToIntMinutes(hours,minutes);
}
public static double hoursMinutesToDecHours(int hours, int minutes) {
    return hours+minutes/60.0;
}
public static int hoursMinutesToIntMinutes(int hours, int minutes) {
    minutes=hours*60+minutes;
    return minutes;
}
@Override
public String toString() {
    return String.format("%2d:%02d",hours, minutes);
}
@Override
public int hashCode() {
    return Objects.hash(hours, minutes);
}
```

boolean equals

```
    @Override
     public boolean equals(Object obj) {
         if (this == obj) {
             return true;
         } else if (!(obj instanceof WorkoutDuration)) {
             return false;
         } else {
             WorkoutDuration other = (WorkoutDuration) obj;
             return Objects.equals(hours, other.hours)
                     && Objects.equals(minutes, other.minutes);
             //return hours == other.hours
             //         && minutes == other.minutes;
             //return hours*60+minutes == other.hours*60+other.minutes;
         }
     }
}

public class Main {
...
    static public void incrWD(WorkoutDuration w1, int m) {
        w1.addMinutes(m);
        //w1=new WorkoutDuration(m*2);
        System.out.println("w1: " + w1);
        m=45;
    }
```

```
public class Main {
    public static void main(String[] args) {
        WorkoutDuration d1 = new WorkoutDuration();
        WorkoutDuration d2 = new WorkoutDuration(15);

        System.out.println("d2: " + d2);
        d2.addHours(1.5);
        System.out.println("d2: " + d2);

        System.out.println("d1: " + d1);
        int minutesPlus= 45;
        incrWD(d1,minutesPlus);
        System.out.println(minutesPlus);
        System.out.println("d1: " + d1);
        d1.addHours(0.5);
        System.out.println("d1==d2 " + d1.equals(d2));
        d1.addMinutes(30);
        System.out.println("d1: " + d1);
        System.out.println("d1==d2 " + d1.equals(d2));
```

```
d2:  0:15
d2:  1:45
d1:  0:00
w1:  0:45
45
d1:  0:45
d1==d2 false
d1:  1:45
d1==d2 true
```

`Static` variables

- *Static* variables are variables that belong to a class as a whole, and not to each object separately
  - There is only one copy of the static variable for each class, as opposed to instance variables where each object has its own copy
- All objects of the class can read and change the value of a static variable
- The static method cannot access an instance variable, but it can access a static variable
- A static variable is declared as an instance variable by adding the static modifier

```
private static int myStaticVariable;
```

- Static variables can be declared and initialized at the same time

```
private static int myStaticVariable = 0;
```

- If not explicitly initialized, they will automatically be assigned a predefined value

`Static` variables

- Static variables should always be defined as private, unless used to define a constant
  - The value of a static defined constant cannot be changed, so it is safe to set it as **`public`**
  - In addition to **`static`**, the declaration of a constant must include the modifier **`final`**, which means that its value cannot be changed

```
public static final int BIRTH_YEAR = 1954;
```

- When accessing such a defined constant outside the class, the name of the class is used instead of a specific object

```
int year = MyClass.BIRTH_YEAR;
```

Strings are a reference type, but...

- The strings are a special case!
  Strings are immutable objects!
- Any change to a string means creating a new one, and therefore they do not have the typical properties of reference objects.

Comparing strings

- `s1 == s2` compares references (not content)
- `s1.equals(s2)` compares content
  - returns Boolean
- `s1.CompareTo(s2)`
  - returns int (-1,0,+1)
  - compares lexicographically
- `compareToIgnoreCase()`

Using == with the String class

- The equality comparison operator (==) can correctly compare values of variables of primitive types
- However, comparing two objects of any class such as objects of class String, == compares whether those two objects have the same memory address
- To compare two objects, it is necessary to use the **`equals`** method
- **`string1.equals(string2)`**
- **`string1.equalsIgnoreCase(string2)`**

Lexicographic and alphabetical comparison

- *Lexicographic* sorting is the same as ASCII sorting, including letters, numbers, and other characters
  - All uppercase and lowercase letters are alphabetically ordered, but all uppercase letters precede lowercase letters
  - If `s1` and `s2` are two variables of type `String` then `s1.compareTo(s2)` returns a negative number if `s1` is lexicographically before `s2`, returns zero if both values are equal, and returns a positive number if `s2` is lexicographically before `s1`
- Alphabetical comparison is enabled by the `compareToIgnoreCase` method

String example

new ...
not required

```
String s1 = "qwerty";
String s2 = "qwerty";
String s3;
s3=s1;
String s4 = new String("qwerty");
//String s4 = String.copyValueOf(s1.toCharArray());

System.out.println("(s1 == s2) = " + (s1 == s2));
System.out.println("(s1 == s3) = " + (s1 == s3));
System.out.println("(s1 == s4) = " + (s1 == s4));

System.out.println("s1.equals(s2) = " + s1.equals(s2));
System.out.println("s1.equals(s4) = " + s1.equals(s4));
```

```
(s1 == s2) = ?
(s1 == s3) = ?
(s1 == s4) = ?
s1.equals(s2) = ?
s1.equals(s4) = ?
```

Wrapper Classes

- *Wrapper classes* provide a class type for the corresponding primitive types (wraps the primitive type inside an object)
  - This allows primitive values to be treated as standard Java objects
  - The wrapper classes for the primitive data types: byte, short, int, long, float, double, char and boolean are (in order) Byte, Short, Integer, Long, Float, Double, Character and Boolean
  - Wrapper classes also contain additional useful constants and static methods

Wrapper Classes

- *Boxing*: the process of transforming a value of primitive data type into an object of the corresponding data class
  - To convert a value of a primitive data type into an "equivalent" value of a class-type, one needs to create an object of the corresponding wrapper class using the primitive value as an argument
  - The newly created object will contain a copy of the primitive value
  - Unlike other classes, wrapper classes do not have a default constructor

```
Integer integerObject = new Integer(42);
Integer integerObject = Integer.valueOf(42);
```

'Integer(int)' is deprecated since version 9 and marked for removal

Wrapper Classes

- *Unboxing*: the process of transforming an object from a wrapper class into a value of the corresponding primitive data type
- Methods that convert objects from wrapper classes: **`Byte`**, **`Short`**, **`Integer`**, **`Long`**, **`Float`**, **`Double`** and **`Character`** in their corresponding primitive types are (in order) **`byteValue`**, **`shortValue`**, **`intValue`**, **`longValue`**, **`floatValue`**, **`doubleValue`**, and **`charValue`**
  - These methods have no arguments

```
int i = integerObject.intValue();
```

Wrapper Classes

- Starting with version 5.0, Java can automatically do boxing and unboxing
- Instead of creating a wrapper class object using the new operator (as shown earlier), this is done by automatically casting types:

```
Integer integerObject = 42;
```

- And similarly for the reverse

```
int i = integerObject;
```

Constants and Static Methods in the Wrapper Classes

- Wrapper classes provide static methods for converting a correctly formed string (representing a certain number) into a number of the corresponding data type
- The methods **`Integer.parseInt`**, **`Long.parseLong`**, **`Float.parseFloat`** и **`Double.parseDouble`** perform this conversion for the primitive data types **`int`**, **`long`**, **`float`** and **`double`**
- The wrapper classes also provide static methods for converting a numerical value into a corresponding string representation
- For example,

  `Double.toString(123.99);`

  returns a string with value **`"123.99"`**

```
int x = Integer.parseInt("9");
double c = Double.parseDouble("5.6");
int b = Integer.parseInt("ABC",16);
```

- The **`Character`** class provides a comprehensive set of static methods for character manipulation and classification

Character class static utility methods

- `isLetter(char ch)`: Checks if a character is a letter.
- `isDigit(char ch)`: Checks if a character is a digit.
- `isUpperCase(char ch)`: Checks if a character is an uppercase letter.
- `isLowerCase(char ch)`: Checks if a character is a lowercase letter.
- `isWhitespace(char ch)`: Checks if a character is a whitespace character (space, tab, newline, etc.).
- `toUpperCase(char ch)`: Converts a character to its uppercase equivalent.
- `toLowerCase(char ch)`: Converts a character to its lowercase equivalent.
- `forDigit(int digit, int radix)`: Returns the character representation of a digit in a specified radix.
- `getNumericValue(char ch)`: Returns the int value that the specified Unicode character represents.

Mutable and immutable classes

- A class that contains public methods that can modify its data members is called a *mutable class*, and its objects are consequently called *mutable objects*
  - Do not write methods that return a mutable object
  - Instead, use a copy constructor to return a reference to a completely independent copy of the mutable object

Deep Copy vs. Shallow Copy

- *deep copy* - a copy of an object that does not share a common reference with the original (with one exception)
  - Exception: Common references to immutable objects are allowed only
- A deep copy creates a completely new, independent copy of the original object. It recursively copies all primitive field values and also creates new, independent copies of all nested objects
- Any copy that is not a *deep copy* is called a *shallow copy*
  - This type of copy can contribute to serious data privacy breaches (privacy leaks)

Arrays in Java

- In Java, an array type is a reference data type that represents a fixed-size, ordered collection of elements of the same data type.
- Arrays in Java are objects
- Array can be viewed as single object but also as a collection of indexed variables

```
double[] a;
a = new double[10];
```

Arrays

- Indexes start at `0` as in C / C++ and are handled similarly to C / C++ (may be declared differently)
- For declarations, angle brackets [] can be inserted either
  - After the type `int[] table; or`
  - After the identifier `int table[];`
- There is no need to specify the length in the declaration

```
int[] numbers; // declare numbers as an int array of any size
```

- Arrays are still fixed size, but the length is specified in the construction

```
numbers = new int[10];  // numbers is a 10-element array
numbers = new int[20];  // now it's a 20-element array
```

Arrays – Examples 1

```
int [] numbers;
numbers = new int[5];
numbers[0]=1;
for (int i=0; i < numbers.length; i++) {
    System.out.print(numbers[i]+ " ");
}
System.out.println();
numbers = new int[] { 0, 1, 2, 3 };
for (int x : numbers) {
    System.out.print(x+ " ");
}
System.out.println();
```

Built-in Public final Field

```
1 0 0 0 0
0 1 2 3
```

Arrays – Examples 2

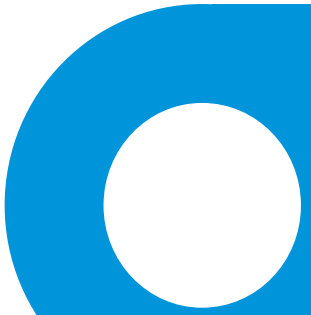
```
String names [ ][ ] =  new String [][] {
        { "Mr.", "Mrs.", "Ms."},
        { "Smith", "Jones"}
};
for (int i=0; i < names.length; i++) {
    for (int j = 0; j < names[i].length; j++) {
        System.out.print(names[i][j] + ", ");
    }
    System.out.println();
}
// System.out.print(names[1][2]);
// would give error: Index 2 out of bounds for
length 2
```

```
Mr., Mrs., Ms.,
Smith, Jones,
```


Questions?



"Ss. Cyril and Methodius" University in Skopje
FACULTY OF COMPUTER
SCIENCE AND ENGINEERING



Inheritance

Introduction to Inheritance

- *Inheritance* is one of the basic techniques of object-oriented programming
- The derived class automatically inherits all instance variables and methods defined in the base class, and can additionally define new instance variables and methods
- Inheritance is particularly important because it allows the use of pre-written code without the need to copy it into derived classes
- All common methods and variables are placed in the superclass (base, parent class)
- When some of them needs to be changed, the change will be performed in one place only, in the superclass

Inheritance

- Inheritance allows us to define a base class and other classes inherited from it
- The code in the methods in the base class can be used by its objects, as well as the objects from derived classes
- If we need a different behavior for in some of the derived classes, we can change the definition of the method inherited from the base class in the particular derived class.
- Those changes apply to the software written using the object of that derived class only (and not the objects of the base class or other classes derived from the base class)

Deriving a subclass in Java

- In Java, to implement inheritance the **extends** keyword is used
- A class that **extends** another class is said to be a **subclass**
-
- Syntax, example:
- `public class ThereeDShape extends Shape`

Overriding

- Members of subclasses may require their own (different) version of the methods they inherit.
- A subclass can change (redefine) its member methods. This is called *overriding*.
- A subclass defines its own version of the method inherited from the superclass (a method with the same name and signature as in the superclass)

In JAVA, all methods

(except static, private and final)

are VIRTUAL

Method Overriding

- In Java, when a class (the **subclass**) extends another class (the **superclass**), it automatically inherits the superclass's accessible fields and methods. The source code for these inherited members is not explicitly copied into the subclass file; rather, it is implicitly available and can be used as if it were part of the subclass. This promotes code reuse.
- While a subclass inherits methods from its superclass, it can provide a specific implementation for an inherited method. This is called **method overriding**. It allows a subclass to offer a specialized version of a method that is already provided by its superclass.
- To override a method, the subclass must define a method with the **same signature** (same name and same number and type of parameters) as the method in the superclass. It is a best practice to use the @Override annotation to instruct the compiler that you intend to override a superclass method. This helps prevent errors, such as accidentally misspelling the method name or using different parameters.

Rules for Return Types in Method Overriding

- When a method in a subclass overrides a method in its superclass, it must adhere to strict rules regarding the return type.
  - **Primitive Types:** If the original method returns a primitive type (e.g., int, double, boolean), the overriding method **must return the exact same** primitive type.
  - **Class Types (Covariant Return Types):** If the original method returns a class type (e.g., Object, String, or any other object), a special rule applies. This rule is known as having a **covariant return type**.
- A covariant return type means the overriding method can return a **subtype** of the original method's return type. In other words, the new return type can be the same class or any class that inherits from it (a subclass).
  - This feature was introduced in **Java 5.0** and significantly increases the flexibility of method overriding.

Covariant return type

- Considering the following base class:

```
public class BaseClass
{ . . .
  public Employee getSomeone(int someKey)
  . . .
```

- The following is allowed in Java 5.0:

```
public class DerivedClass extends BaseClass
{ . . .
  public HourlyEmployee getSomeone(int someKey)
  . . .
```

- where `HourlyEmployee extends Employee`

Rules for Access Modifiers in Method Overriding

- In Java, when overriding a method, the access modifier of the method in the subclass must be **the same or less restrictive** than the one in the superclass. You can widen the access, but you cannot make it more restrictive.
  - A `protected` method in the superclass can be made `protected` or `public` in the subclass.
  - A `public` method in the superclass **must** remain `public` in the subclass.
  - A `private` method **cannot be overridden** at all. `private` members are not visible to subclasses, so defining a method with the same signature in a subclass **creates a new method**, not an override.

Access Modifiers in Overridden Methods

- For the following method declaration in the main class:

```
protected void doSomething()
```

- In the derived class, the declaration is allowed to be changed to:

```
public void doSomething()
```

- The opposite is not allowed

```
public void doSomething()
private void doSomething()// it's illegal
```

Overriding in Java

- To correctly override a method, the method in the subclass must have the exact same **method signature** as the method in the superclass.
  - In Java, a method's signature is defined by its **name** and its **parameter list** (the number, type, and order of its parameters). The return type and access modifier are not part of the signature but have their own rules, as discussed previously.
- **Static Methods Cannot Be Overridden**
  - If a subclass defines a static method with the same signature as a static method in its superclass, this is known as **method hiding**, not overriding. The method that gets called depends on the type of the reference variable, not the type of the object.

Overriding in Java

- **Fields (Instance Variables) Cannot Be Overridden**
  - Fields (variables) in Java are not subject to polymorphism and cannot be overridden. If a subclass declares a field with the same name as a field in its superclass, the subclass's field **hides** the superclass's field. This is known as **variable hiding** or shadowing.
    - Both variables still exist.
    - The variable that is accessed depends on the reference type used.
- The Consequence of Overriding: Dynamic Method Dispatch
  - The implementation in the subclass **replaces** the implementation inherited from the superclass.
  - When you call an overridden method on an object, the Java Virtual Machine (JVM) uses a mechanism called **dynamic method dispatch**. At runtime, the JVM looks at the actual type of the object, not the type of the reference variable pointing to it, to determine which version of the method to execute. This ensures that the most specific implementation (the one in the subclass) is always chosen.

Example

```
public class T {
   public void m() { … }
}
public class S extends T {
   public void m() { … }
}
T t = new T();
S s = new S();
t.m(); //   calls the method m from class T
s.m(); //   calls the method m from class S
```

```
T r = new S();
r.m(); // ???
S u = new T();
u.m(); // ???
```

S.m()

error

Binding in Java: Static vs. Dynamic

- The process of connecting a method call (the line of code invoking the method) to the specific method implementation (the definition) that will actually execute is called *Binding*.
- Java uses two distinct types of binding, based on when this link is established:

1. **Static Binding** (Compile-Time Binding / Early Binding)
2. **Dynamic Binding** (Runtime Binding / Late Binding)

Static Binding

- Static Binding occurs when the compiler is able to resolve the link between the method call and the method definition before the program executes (i.e., during compilation).
- The binding decision is based solely on the **type of the reference variable**, not the actual object it holds.
- In Java, Static Binding applies to methods that cannot participate in overriding or runtime polymorphism, because their structure is fixed and known.
- Applies to:
  1. `static` **methods**: They belong to the class, not the object.
  2. `private` **methods**: They are not accessible outside the class, so they cannot be overridden.
  3. `final` **methods**: Their implementation is fixed and cannot be changed by subclasses.
  4. **Overloaded methods**: Method overloading is resolved based on the argument list available at compile time.

Dynamic Binding

- Dynamic Binding occurs when the link between the method call and the method definition cannot be determined until the program is executing (at runtime) by the Java Virtual Machine (JVM).
- The binding decision is based on the **actual object type** (the object instance) being referenced, which may differ from the declared reference type. This mechanism is the core enabler of **Runtime Polymorphism**. Dynamic Binding is essential for **Method Overriding**.
- Applies to: Any non-private, non-static, non-final instance method in a class hierarchy that has been overridden.

Dynamic binding

- When you call an overridden method on an object, the Java Virtual Machine (JVM) uses a mechanism called **dynamic method dispatch**. At runtime, the JVM looks at the actual type of the object, not the type of the reference variable pointing to it, to determine which version of the method to execute. This ensures that the most specific implementation (the one in the subclass) is always chosen.

```
class Student {
  public int minCredits() { return 12; }
  …
}
class GradStudent extends Student {
  public int minCredits() { return 9; }
  …
}

Student s = new Student();
// …
int x=s.minCredits(); // which value will be assigned to x?
```

12

Dynamic binding

- When you call an overridden method on an object, the Java Virtual Machine (JVM) uses a mechanism called **dynamic method dispatch**. At runtime, the JVM looks at the actual type of the object, not the type of the reference variable pointing to it, to determine which version of the method to execute. This ensures that the most specific implementation (the one in the subclass) is always chosen.

```
class Student {
  public int minCredits() { return 12; }
  …
}
class GradStudent extends Student {
  public int minCredits() { return 9; }
  …
}

Student s = new GradStudent();
// …
int x=s.minCredits(); // which value will be assigned to x?
```

9

Late Binding

- As a result of the late binding, a method can be written in the base class, even if parts of it are not yet defined (left to be specified in the derived classes)

Late Binding Example

```
Scanner scanner = new Scanner(System.in);
System.out.print("Enter student level (1 for Undergraduate, 2 for Graduate): ");
int choice = scanner.nextInt();
// The compiler only knows this variable can hold a Student reference:
Student student;

// The actual object assigned depends entirely on user input at runtime:
if (choice == 1) {
    student = new Student();
} else {
    student = new GradStudent();
}

// Late binding happens here:
// The compiler cannot predict whether choice is 1 or 2.
// The JVM resolves and calls the correct minCredits() version at runtime.
int credits = student.minCredits();
System.out.println("Minimum required credits for this student: " + credits);
scanner.close();
```

Late Binding example

- `class DiscountSale extends Sale`
- Definirtion of `bill()` in `Sale` class:

```
public double bill( )
{
  return price;
}
```

- Definirtion of `bill()` in `DiscountSale` class:

```
public double bill( )
{
  double fraction = discount/100;
  return (1 - fraction) * getPrice( );
}
```

Late Binding example

- The following code segment:

```
   . . .
Sale simple = new sale("floor mat", 10.00);
DiscountSale discount = new
            DiscountSale("floor mat", 11.00, 10);
   . . .
```

What will return: **`discount.lessThan(simple) ?`**

Overriding vs. Overloading in Java

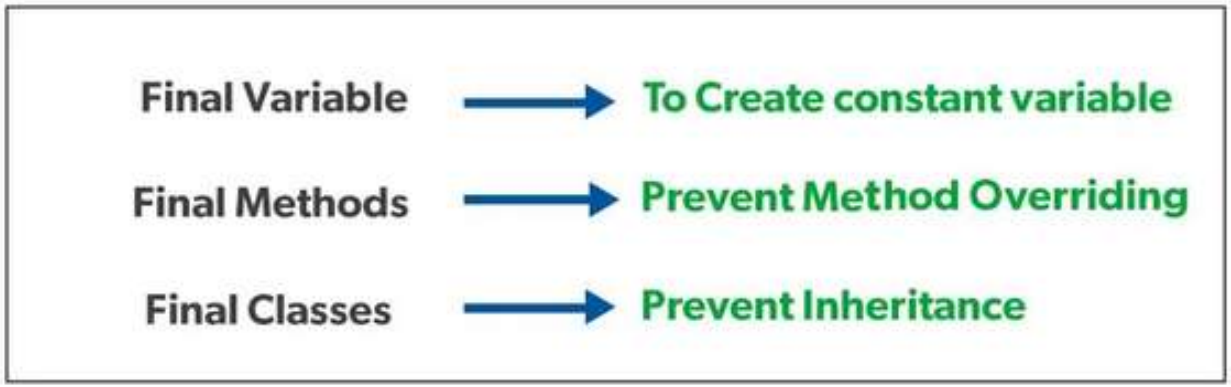
- In Java, **method overriding** and **method overloading** are two forms of polymorphism, but they are fundamentally different.
- **Method Overriding (Runtime Polymorphism)**
  - a subclass (the derived class) provides a specific implementation for a method that is already defined in its superclass (the base class).
  1. **Relationship**: It occurs between a superclass and a subclass (an is-a relationship).
  2. **Method Signature**: The method in the subclass must have the exact same name, number of parameters, and type of parameters as the method in the superclass.
  3. **Purpose**: To change the behavior of an existing method for the subclass.
  4. **Annotation**: It is a best practice to use the `@Override` annotation, which tells the compiler you intend to override a method. The compiler will produce an error if you make a mistake (e.g., misspell the name or change the parameters).

Overriding vs. Overloading in Java

- **Method Overloading (Compile-Time Polymorphism)**
  - allows you to define multiple methods with the same name within the same class, as long as their parameter lists are different.
  1. **Relationship**: It can occur within the same class or in a subclass.
  2. **Method Signature**: The methods must have the **same name** but a **different parameter list** (either a different number of parameters, different types of parameters, or a different order of types).
  3. **Purpose**: To create methods that perform similar operations but work with different input data.
  4. **Annotation**: Overloaded methods can have different return types, but the return type alone is not sufficient to distinguish them.

`final` modifier

- When the `final` modifier is used on a method, it declares that this method's implementation is complete and cannot be changed or overridden by any subclass.
- When the `final` modifier is used on a class, it declares that the class is complete and cannot be extended (it cannot be a superclass).



`super` constructor

- The derived class uses the constructor of the base class to initialize all data members inherited from the base class
- To explicitly invoke the constructor of the base class, a special syntax is used:

```
public derivedClass(int p1, int p2, double p3)
{
  super(p1, p2);
  instanceVariable = p3;
}
```

  - In the previous example, `super(p1, p2);` is a call for a base-class constructor

```
class derivedClass extends baseClass …
```

`super` constructor

- The call to `super` must always be the first statement in the derived constructor definition
- An instance variable cannot be used as an argument to a call of `super`
- If the constructor of the derived class does not include a call to `super` (or call to `this`), then the default constructor of the base class is automatically called
  - This can result in an error if a constructor with no arguments is not defined in the base class
- If the inherited variables of an instance need to be initialized to specified values, and there is constructor in the base class for this, then an explicit call to `super` should be used

`this` constructor

- In the definition of a class constructor, **`this`** can be used to call another version of the constructor of the same class
  - The mode of use is the same as with the super constructor
- A constructor can only contain either `super()` or `this()`, but not both (and it must be the absolute first statement in a constructor).

`this` constructor

```
public HourlyEmployee()
{
   this("No name", new Date(), 0, 0);
}
```

- The previous constructor calls the constructor with the following signature:

```
public HourlyEmployee(String theName,
  Date theDate, double theWageRate, double
  theHours)
```

Type Relationships in Inheritance

- An object of a derived (subclass) has multiple types.
- It is an instance of its own class, but also of all its ancestor (super) classes.
- Therefore, an object of a derived class can be assigned to a variable of any ancestor type (upcasting).
- The reverse is not allowed without an explicit cast (downcasting).

Encapsulation and Inheritance

- An **instance variable** declared as `private` in a base class cannot be accessed directly from any other class — **not even from a derived class.**
- **Private instance variables** can be accessed **only by methods defined within the same class.**
- An object of the `HourlyEmployee` class can access the private variable `hireDate` of its base class only through the public methods `getHireDate()` and `setHireDate()`.

Private Methods and Inheritance

- **Private methods** in a base class behave like private variables in terms of **access control**.
- They are **completely inaccessible** from outside the class — including from derived classes — unless called indirectly.
- Indirect access is possible when a public method of the base class calls the private method, and that public method is then invoked by an object of the derived class.
- Private methods are typically used as helper (utility) methods. If a method is intended to be used more broadly, it should be declared as `public` rather than `private`.

Java Access Modifiers

Java access modifiers control the visibility and accessibility of classes, fields, methods, and constructors. They are crucial for implementing encapsulation and maintaining modularity in your code.

There are four main access modifiers in Java:

- `private`:
  - Accessibility: Accessible only within the class where it is declared.
  - Usage: Typically used for fields and helper methods that should not be directly accessed from outside the class, promoting data hiding and encapsulation.

Java Access Modifiers

- **Default** (Package-Private):
  - Accessibility: Accessible only within the same package. No explicit keyword is used; it's the default if no other access modifier is specified.
  - Usage: Useful for classes, fields, and methods that need to be shared among classes within the same package but not exposed to other packages.
- `protected`:
  - Accessibility: Accessible within the same package and by subclasses, even if the subclasses are in different packages.
  - Usage: Designed for members that are intended to be inherited and potentially overridden by subclasses, while still maintaining some level of access control.
- `public`:
  - Accessibility: Accessible from anywhere, including within the same class, same package, different packages, and by subclasses or non-subclasses.
  - Usage: Used for classes, fields, methods, and constructors that need to be widely available and form the public interface of a component.

`Protected` and `Package` Access

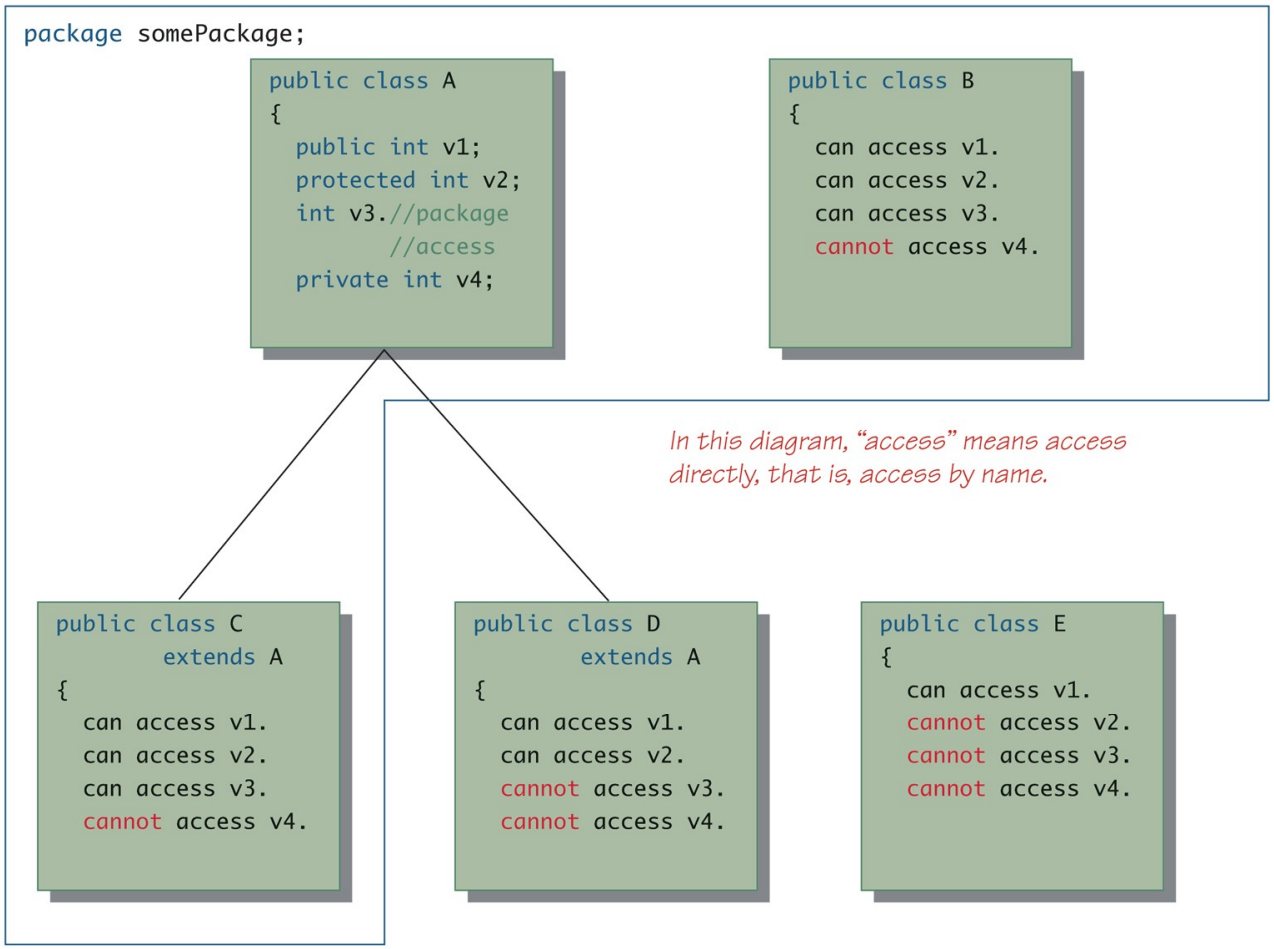
- If a method or instance variable is declared as **`protected`** (instead of **`public`** or **`private`**), it can be accessed:
  - Within the same class,
  - From any subclass, and
  - From any class within the same package.
- The **`protected`** modifier provides weaker protection than **`private`**.
  - It allows direct access by a programmer defining a derived class.

`Protected` and `Package` Access

- If **no access modifier** is specified, the member has **package** (default or friendly) access.
- Members with **package access** can be accessed by **any class within the same package**, but **not from outside the package**.
- **Package access** is **more restrictive** than `protected`, giving the programmer greater control over class design.
- Whoever **controls the package** directory also controls **package-level access**.

Access Modifiers

Display 7.9 Access Modifiers



Quiz...

Classes **A** and **B** are located in different packages.

Class **B** extends class **A**, and class **A** has a variable **n** declared as **protected**.

Which of the following statements are true?

- ☑ A method in class **B** can directly access **n**. (inheritance)
- ☑ A method in class **B** can create a local object of class **B** and access **n** through that object. (same subclass)
- ☒ A method in class **B** can create an object of class **A** and access **n** through that object. (outside the package)

Static variables are inherited

- Static variables of the base class are inherited in all derived classes
- The modifiers **`public`**, **`private`**, and **`protected`**, and package access have the same meaning for static variables as they do for instance variables

Access to an overridden method

- A method of the base class may be called within the definition of an overridden method in the derived class
  - The overridden method is invoked using the keyword super

```
public String toString()
{
   return (super.toString() + "$" + wageRate);
}
```

- When using an object of the derived class outside the class definition, there is no way to invoke the original method of the base class

The `Object` class

- The `Object` class is located in the `java.lang` package, which is automatically imported.
- This allows methods to be written with parameters of type `Object`.
- A parameter of type `Object` can be replaced with an object of any class.
- The `Object` class defines several methods that every Java class inherits, such as `equals()` and `toString()`.
- Every object inherits these methods from some ancestor class.
- These inherited methods are typically overridden to provide behavior that fits the specific class.

class Employee

```
class Employee {
        private String name;
        private int id;

        public Employee(String name, int id) {
            this.name = name;
            this.id = id;
        }

        // Overriding toString() from Object
        @Override
        public String toString() {
            return "Employee[name=" + name + ", id=" + id + "]";
        }

        // Overriding equals() from Object
        @Override
        public boolean equals(Object obj) {
            if (this == obj) return true;                    // same reference
            if (!(obj instanceof Employee)) return false; // type check
            Employee other = (Employee) obj;
            return id == other.id && name.equals(other.name);
        }
    }
```

class Employee

```
public class Main {
	public static void main(String[] args) {

		int n=3;
		Employee emp1 = new Employee("John", 1);
		Employee emp2 = new Employee("John", 1);
		Employee emp3 = new Employee("Jane", 2);
		System.out.println(emp1);

		System.out.println(emp1.equals(emp1));
		System.out.println(emp1.equals(emp2));
		System.out.println(emp1.equals(emp3));
		System.out.println(emp1.equals(n));
	}
}
```

```
Employee[name=John, id=1]
true
true
false
false
```

Checking an Object's Type `getClass` vs. `instanceof`

- Both the **`instanceof`** operator and the **`getClass()`** method are used to get information about an object's type at runtime.
- **`instanceof`** asks: "*Is this object an instance of this class, or any of its subclasses?*" (An "is-a" relationship).
- **`getClass()`** asks: "*What is the exact, original class that this object was created from? *"
- The **`instanceof`** operator is a binary operator used to test if an object is of a given type. It evaluates to either **`true`** or **`false`**. It is the preferred way to check for type compatibility in a class hierarchy.
- The **`getClass()`** method is defined in the **`java.lang.Object`** class, so every object in Java has it. It returns a **`Class<?>`** object, which represents the exact runtime type of the object.
- The use of both is valid in **`equals()`** – it depends on what we would like to achieve

class Manager extends Employee

```
class Manager extends Employee {
        private String department;

        public Manager(String name, int id, String department) {
            super(name, id);
            this.department = department;
        }

        // equals() using getClass() (requires exact same class)
        @Override
        public boolean equals(Object obj) {
            if (this == obj) return true;
            if (obj == null || getClass() != obj.getClass()) return false;
            Manager other = (Manager) obj;
            // Compare fields from both Employee (via super) and Manager
            return super.equals(other) && department.equals(other.department);
        }
        @Override
        public String toString() {
            return "Manager" + super.toString() + ", department=" + department + "]";
        }
    }


    Employee e1 = new Employee("Alice", 101);
    Manager m1 = new Manager("Alice", 101, "Sales");

    System.out.println(e1.equals(m1)); // true with instanceof, false with getClass()
    System.out.println(m1.equals(e1)); // false (getClass requires exact match)
```

PreciseWorkoutDuration

```
public class PreciseWorkoutDuration extends WorkoutDuration {
    protected int seconds;
    PreciseWorkoutDuration() {
        super();
        seconds = 0;
    }
    PreciseWorkoutDuration(double m) {
        this(0, 0, (int) (m * 60));
    }
    PreciseWorkoutDuration(int h, int m) {
        this(h,m,0); //super(h,m); seconds = 0;
    }
    PreciseWorkoutDuration(int h, int m, int s) {
        seconds = s % 60;
        minutes = m + s / 60;
        hours = h + minutes / 60;
        minutes %= 60;
    }
```

Invoking a constructor of the base class

Calling another version of the constructor of the current class

```
public void addSeconds(int s) {
    seconds += s;
    minutes += seconds / 60;
    seconds %= 60;
    hours += minutes / 60;
    minutes %= 60;
}
@Override
public void addMinutes(double m) {
    addSeconds((int)(m*60));
}
@Override
public double getHours() {
    return hours + (minutes + seconds / 60.0) / 60;
}
/* Cannot change return type
    public double getMinutes() {
        return hours*60+minutes+seconds/60.0;
    }
*/
public int getSeconds() {
    return (hours * 60 + minutes) * 60 + seconds;
}
```

```java
@Override
public String toString() {
    return super.toString() + String.format(":%02d", seconds);
}
@Override
public int hashCode() {
    return Objects.hash(hours, minutes, seconds);
}
@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    } else if (obj instanceof PreciseWorkoutDuration) {
        PreciseWorkoutDuration other = (PreciseWorkoutDuration) obj;
        return Objects.equals(hours, other.hours)
                && Objects.equals(minutes, other.minutes)
                && Objects.equals(seconds, other.seconds);
    } else if (obj instanceof WorkoutDuration) {
        WorkoutDuration other = (WorkoutDuration) obj;
        return Objects.equals(hours, other.hours)
                && Objects.equals(minutes, other.minutes)
                && seconds < 30;
    } else {
        return false;
    }
}
```

```
public class Main {
    public static void main(String[] args) {
. . .

    PreciseWorkoutDuration pd1 = new PreciseWorkoutDuration(1, 45);
    System.out.println("pd1: " + pd1);
    pd1.addSeconds(25);
    System.out.println("pd1: " + pd1);
    System.out.println("d1==pd1 " + d1.equals(pd1));
    System.out.println("pd1==d1 " + pd1.equals(d1));
    pd1.addSeconds(10);
    System.out.println("d1==pd1 " + d1.equals(pd1));
    System.out.println("pd1==d1 " + pd1.equals(d1));

    System.out.println("================");
    WorkoutDuration pd2 = new PreciseWorkoutDuration(1, 45, 35);
    System.out.println("pd2: " + pd2);
    System.out.println("d1==pd2 " + d1.equals(pd2));
    System.out.println("pd2==d1 " + pd2.equals(d1));

}
```

```
pd1:  1:45:00
pd1:  1:45:25
d1==pd1 true
pd1==d1 true
d1==pd1 true
pd1==d1 false
================
pd2:  1:45:35
d1==pd2 true
pd2==d1 false
```

Upcasting and Downcasting

- *Upcasting* is when an object of a derived class is assigned to a variable of the base class
- `Sale saleVariable; // Base class`

```
DiscountSale discountVariable = new
    DiscountSale("paint", 15,10); // Derived class
saleVariable = discountVariable; // Upcasting
System.out.println(saleVariable.toString());
```

- Due to late binding, **`toString`** uses the definition given in the **`DiscountSale`** class

Upcasting and Downcasting

- *Downcasting* is when casting is done from a base class to a derived class
- Downcasting should be performed very carefully
  - In many cases, it doesn't make any sense or it's illegal:

```
discountVariable =                    // will produce
     (DiscountSale)saleVariable;// run-time error
discountVariable = saleVariable // will produce
                                      // compiler error
```

  - However, downcasting is sometimes necessary, e.g. In the `equals` method:

```
Sale otherSale = (Sale)otherObject; // downcasting
```

  - ... once we make sure that the object of the base class **actually references** to an object of the given class

`clone` method

- Each object inherits the **`clone`** method from the Object class
  - The clone method takes no arguments
  - It is assumed that it should return a **deep copy** of the calling object
  - The implementation of making the deep copy of an object (overriding the clone method in a particular class) is responsibility of the programmer (the default implementation performs only a shallow copy)

`clone` method

- The signature of the **`clone`** method defined in the **`Object`** class is:

```
protected Object clone()
```

- The signature of the **`clone`** method that overrides the method inherited from the **`Object`** class may differ in:
  - Access specifier. Access permission can be changed from **`protected`** to **`public`**. Typically, to make it accessible to other classes or for custom cloning logic, it needs to be promoted to public.
  - The type being returned as a result from **`Object`** to the type of the actual object being cloned
  - This is an example of a **covariant return type**

`clone` method

- If a class has a copy constructor, the **`clone`** method can use the *copy* constructor to create a copy and return it

```
public Sale clone()
{
  return new Sale(this);
}
```

- Although the copy constructor and the clone method for a class in principle do the same job, they both have some limitations:
  - Copy Constructor - Lack of polymorphism
  - Clone - Shallow copy by default, No constructor invocation

class Sale

```
/**
 Class for a simple sale of one item with no tax, discount, or other adjustments.
 Class invariant: The price is always nonnegative; the name is a nonempty string.
 */
public class Sale
{
    private String name; //A nonempty string
    private double price; //nonnegative
    public Sale()
    {
        name = "No name yet";
        price = 0;
    }
    /**
     Precondition: theName is a nonempty string; thePrice is nonnegative.
     */
    public Sale(String theName, double thePrice)
    {
        setName(theName);
        setPrice(thePrice);
    }
```

class Sale

```
    public Sale(Sale originalObject)
    {
        if (originalObject == null)
        {
            System.out.println("Error: null Sale object.");
            System.exit(0);
        }
//else
        name = originalObject.name;
        price = originalObject.price;
    }
    public static void announcement()
    {
        System.out.println("This is the Sale class.");
    }
    public double getPrice()
    {
        return price;
    }
```

class Sale

```
/**
 Precondition: newPrice is nonnegative.
 */
public void setPrice(double newPrice)
{
    if (newPrice >= 0)
        price = newPrice;
    else
    {
        System.out.println("Error: Negative price.");
        System.exit(0);
    }
}
public String getName()
{
    return name;
}
```

class Sale

```
 /**
 Precondition: newName is a nonempty string.
 */
public void setName(String newName)
{
    if (newName != null && newName != "")
        name = newName;
    else
    {
        System.out.println("Error: Improper name value.");
        System.exit(0);
    }
}
@Override
public String toString()
{
    return (name + " Price and total cost = $" + price);
}
public double bill()
{
    return price;
}
```

class Sale

```
    /*
    Returns true if the names are the same and the bill for the calling
    object is equal to the bill for otherSale; otherwise returns false.
    Also returns false if otherObject is null.
    */
    public boolean equalDeals(Sale otherSale)
    {
        if (otherSale == null)
            return false;
        else
            return (name.equals(otherSale.name)
                    && bill() == otherSale.bill());
    }
    /*
    Returns true if the bill for the calling object is less
    than the bill for otherSale; otherwise returns false.
    */
    public boolean lessThan (Sale otherSale)
    {
        if (otherSale == null)
        {
            System.out.println("Error: null Sale object.");
            System.exit(0);
        }
//else
        return (bill() < otherSale.bill());
    }
```

class Sale

```
    @Override
    public boolean equals(Object otherObject)
    {
        if (otherObject == null)
            return false;
        else if (getClass() != otherObject.getClass())
            return false;
        else
        {
            Sale otherSale = (Sale)otherObject;
            return (name.equals(otherSale.name)
                    && (price == otherSale.price));
        }
    }
    @Override
    public Sale clone()
    {
        return new Sale( this);
    }
}
```

!(otherObject instanceof Sale)

class DiscountSale

```
/**
 Class for a sale of one item with discount expressed as a percent of
 the price, but no other adjustments.
 Class invariant: The price is always nonnegative; the name is a
 nonempty string; the discount is always nonnegative.
 */
public class DiscountSale extends Sale
{
    private double discount; //A percent of the price. Cannot be negative.
    public DiscountSale()
    {
        super ();
        discount = 0;
    }
    /**
     Precondition: theName is a nonempty string; thePrice is nonnegative;
     theDiscount is expressed as a percent of the price and is nonnegative.
     */
    public DiscountSale(String theName, double thePrice, double theDiscount)
    {
        super (theName, thePrice);
        setDiscount(theDiscount);
    }
```

class DiscountSale

```
public DiscountSale(DiscountSale originalObject)
{
    super (originalObject);
    discount = originalObject.discount;
}
public static void announcement()
{
    System.out.println("This is the DiscountSale class.");
}
@Override
public double bill()
{
    double fraction = discount/100;
    return (1-fraction)*getPrice();
}
public double getDiscount()
{
    return discount;
}
```

class DiscountSale

```
/**
Precondition: Discount is nonnegative.
*/
public void setDiscount(double newDiscount)
{
    if (newDiscount >= 0)
        discount = newDiscount;
    else
    {
        System.out.println("Error: Negative discount.");
        System.exit(0);
    }
}
@Override
public String toString()
{
    return (getName() + " Price = $" + getPrice()
            + " Discount = " + discount + "%\n"
            + " Total cost = $" + bill());
}
```

class DiscountSale

```
@Override
    public boolean equals(Object otherObject)
    {
        if (otherObject == null)
            return false;
        else if (!(otherObject instanceof DiscountSale))
            return false;
        else
        {
            DiscountSale otherSale = (DiscountSale)otherObject;
            return (getName().equals(otherSale.getName())
                    && (getPrice() == otherSale.getPrice())
                    && discount == otherSale.discount);
        }
    }
    @Override
    public DiscountSale clone()
    {
        return new DiscountSale( this);
    }
}
```

class CopyingDemo

```
/**
 Demonstrates where the clone method works, but copy constructors do not.
 */
class CopyingDemo {
    public static void main(String[] args) {
        Sale[] a = new Sale[2];
        a[0] = new Sale("atomic coffee mug", 130.00);
        a[1] = new DiscountSale("invisible paint", 5.00, 10);
        int i;
        Sale[] b = badCopy(a);
        System.out.println("With copy constructors:");
        for (i = 0; i < a.length; i++) {
            System.out.println("a[" + i + "] = " + a[i]);
            System.out.println("b[" + i + "] = " + b[i]);
            System.out.println();
        }
        System.out.println();
        b = goodCopy(a);
        System.out.println("With clone method:");
        for (i = 0; i < a.length; i++) {
            System.out.println("a[" + i + "] = " + a[i]);
            System.out.println("b[" + i + "] = " + b[i]);
            System.out.println();
        }
    }
```

class CopyingDemo

```
    /**
     Supposedly returns a safe copy of a. That is, if b is the
     array returned, then b[i] is supposedly an independent copy of a[i].
     */
    public static Sale[] badCopy(Sale[] a)
    {
        Sale[] b = new Sale[a.length];
        for (int i = 0; i < a.length; i++)
            b[i] = new Sale(a[i]);//Problem here!
        return b;
    }

    /**
     Returns a safe copy of a. That is, if b is the
     array returned, then b[i] is an independent copy of a[i].
     */
    public static Sale[] goodCopy(Sale[] a)
    {
        Sale[] b = new Sale[a.length];
        for (int i = 0; i < a.length; i++)
            b[i] = a[i].clone( );
        return b;
    }
}
```

output

```
With copy constructors:
a[0] = atomic coffee mug Price and total cost = $130.0
b[0] = atomic coffee mug Price and total cost = $130.0

a[1] = invisible paint Price = $5.0 Discount = 10.0%
 Total cost = $4.5
b[1] = invisible paint Price and total cost = $5.0


With clone method:
a[0] = atomic coffee mug Price and total cost = $130.0
b[0] = atomic coffee mug Price and total cost = $130.0

a[1] = invisible paint Price = $5.0 Discount = 10.0%
 Total cost = $4.5
b[1] = invisible paint Price = $5.0 Discount = 10.0%
 Total cost = $4.5
```

Abstract method

- An abstract method is like an empty method that needs to be fully defined in the inherited classes
- The abstract method has a declaration to which the modifier abstract is added
- Abstract methods can't be private
- These methods have no body and the declaration ends with ;

```
public abstract double getPay();
public abstract void doIt(int count);
```

Abstract classes

- A class that has at least one abstract method is called an abstract class
- The abstract class includes the **`abstract`** modifier in its definition:

```
public abstract class Employee
{
  private instanceVariables;
  . . .
  public abstract double getPay();
  . . .
}
```

Declared but left unedified here

Abstract classes

- Abstract classes can have abstract or fully defined methods
- If a class that inherits from an abstract class does not implement all abstract methods, then it automatically becomes `abstract` and the abstract modifier must be added to its definition
- A class that does not have abstract methods is called a concrete class

You can't instantiate objects of abstract class

- An abstract class is used to derive specialized classes
- The constructor of an abstract class cannot be used to create an object of an abstract class
- However, the constructor of the inherited class can call the constructor of the abstract class using **`super`**

The abstract class is a type

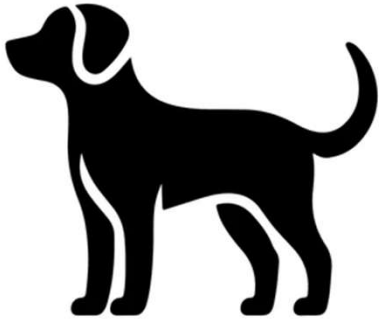
- Although it is not possible to create an object of an abstract class, there is no problem having a parameter of the type of the abstract class
- This allows an object of a derived class to be joined to a parameter of abstract type
- It is also possible for a variable of an abstract data type to refer to an object of a class that directly or indirectly inherits from the abstract class

Abstract classes in JAVA

```
abstract class Animal {
    abstract void makeSound();
    protected String name;
    public Animal(String n) {
        name=n;
    }
    public void eat() {
        System.out.println(name + ": I can eat.");
    }
}
```

Abstract classes in JAVA

```
class Dog extends Animal {
    Dog(String n) {
        super(n);
    }
    // provide implementation of abstract method
    public void makeSound() {
        System.out.println(name+ ": Bark bark");
    }
}
```



Abstract classes in JAVA



```
class Cat extends Animal {
    Cat(String n) {
        super(n);
    }
    // provide implementation of abstract method
    public void makeSound() {
        System.out.println(name+ ": Meow meow");
    }
}
```

Abstract classes in JAVA

```
public class Main {
    public static void main(String[] args) {

        Animal x;




    }
}
```



Abstract classes in JAVA

```
public class Main {
    public static void main(String[] args) {

        Animal x;
        x = new Animal();




    }
}
```



Abstract classes in JAVA

```
public class Main {
    public static void main(String[] args) {

        Animal x;
        x = new Animal();
        x = new Cat("P");

        Animal [] a = new Animal[2];
        a[0] = new Cat("C");
        a[1] = new Dog("D");
    }
}
```

Abstract classes in JAVA

```
public class Main {
    public static void main(String[] args) {

        Animal [] a = { new Cat("Pussy"), new Dog("Puppy") };

        for(Animal x : a) {
            x.eat();
            x.makeSound();
        }

    }
}
```

```
Pussy: I can eat.
Pussy: Meow meow
Puppy: I can eat.
Puppy: Bark bark
```

Interfaces

- An interface in Java defines a contract for classes to adhere to, specifying a set of methods that a class must implement.
- It provides a way to achieve abstraction and **support multiple inheritance of type.**

Interfaces

- The interface is something like an extreme abstract class
  - The interface is **not a class**
  - It is a **type** representative of each class that implements the interface
- The syntax for defining an interface is similar to the syntax for defining a class
  - `interface` instead of `class`
- The interface specifies a **set of method headers** (abstract methods) that any implementing class must satisfy
  - It contains only method headers and definitions of constants (changed since Java 8)
  - It contains no instance variables or complete method definitions (changed since Java 8)
  - Can contain constants (`public static final` fields), but **no instance variables** (interfaces cannot hold object state)
  - Can include complete method bodies via:
    - `default` methods (allow API evolution without breaking implementing classes)
    - `static` utility methods

Interfaces

- The interface and all its methods should be declared public
  - You can't give private, protected, or package access
- When a class implements an interface, all interface methods must be public
- Because the interface is a type, the parameters of the methods can be of type interface
- The parameter will accept any argument from the class that implements the interface

Interface methods are by default **`abstract`** and **`public`**
Interface attributes are by default **`public`**, **`static`** and **`final`**
An interface cannot contain a constructor

Shape interface example

```java
// Define an interface named 'Shape'
interface Shape {
    // Declare abstract methods (implicitly public and abstract)
    void draw();
    double getArea();
}

// Implement the 'Shape' interface in a class named 'Circle'
class Circle implements Shape {
    private double radius;
    public Circle(double radius) {
        this.radius = radius;
    }
    @Override
    public void draw() {
        System.out.println("Drawing a circle with radius: " + radius);
    }
    @Override
    public double getArea() {
        return Math.PI * radius * radius;
    }
}
```

Shape interface example

```
// Implement the 'Shape' interface in a class named 'Rectangle'
class Rectangle implements Shape {
    private double length;
    private double width;

    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    @Override
    public void draw() {
        System.out.println("Drawing a rectangle with length " + length + " and
width " + width);
    }

    @Override
    public double getArea() {
        return length * width;
    }
}
```

Shape interface example

```
// Main class to demonstrate interface usage
class InterfaceExample {
    public static void main(String[] args) {
        // Create objects of classes implementing the interface
        Shape circle = new Circle(5.0);
        Shape rectangle = new Rectangle(4.0, 6.0);
        // Call interface methods on the objects
        circle.draw();
        System.out.println("Area of circle: " + circle.getArea());
        rectangle.draw();
        System.out.println("Area of rectangle: " + rectangle.getArea());
        System.out.println("The circle is " + (isBigger(circle,rectangle) ? "" :
   "not ") + "bigger than the rectangle");
     }
      public static boolean isBigger(Shape a, Shape b) {
         return a.getArea() > b.getArea();
    }
}
```

```
Drawing a circle with radius: 5.0
Area of circle: 78.53981633974483
Drawing a rectangle with length 4.0 and width 6.0
Area of rectangle: 24.0
The circle is bigger than the rectangle
```

```
interface Shape {
...
   static boolean isBigger(Shape a, Shape b) {
      return a.getArea() > b.getArea();
   }
}

…

System.out.println("The circle is " +
(Shape.isBigger(circle,rectangle) ? "" : "not ")
+ "bigger than the rectangle");
```

Interface inheritance

- Abstract classes can also implement one or more interfaces
- A concrete class must implement all the methods it inherits as abstract and the methods of the interface(s) it implements
- An interface can extend another interface (like classes)

Interfaces in JAVA

```
public interface BankAccount {
    void deposit(double amount);
    boolean withdraw(double amount);
    double getBalance();
}
```

DebitAccount

```
public class DebitAccount implements BankAccount {
    private double balance;
    public void deposit(double amount) {
        balance += amount;
    }
    public boolean withdraw(double amount) {
        if (balance >= amount) {
            balance -= amount;
            return true;
        }
        System.out.println("Withdrawal attempt failed.");
        return false;
    }
    public double getBalance() {
         return balance;
    }
    @Override
    public String toString() {
        return String.format("Debit account: Balance = %.2f", balance);
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        BankAccount debit = new DebitAccount();
        debit.deposit(200);
        debit.withdraw(100);
        System.out.println(debit);
        debit.withdraw(200);
    }
}
```

```
Debit account: Balance = 100,00
Withdrawal attempt failed.
```

```
public class CreditAccount implements BankAccount {
    private double balance;
    static private double allowedMinus = 500;
    public void deposit(double amount) {
        balance += amount;
    }
    public boolean withdraw(double amount) {
        if (balance - amount >= -allowedMinus) {
            balance -= amount;
            return true;
        }
        System.out.println("Withdrawal attempt failed.");
        return false;
    }
    public double getBalance() {
        return balance;
    }
    @Override
    public String toString() {
        return String.format("Credit account: Balance = %.2f",
                                                    balance);
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        BankAccount credit = new CreditAccount();
        credit.deposit(300);
        credit.withdraw(100);
        System.out.println(credit);
        credit.withdraw(500);
        System.out.println(credit);
        credit.withdraw(300);
    }
}
```

```
Credit account: Balance = 200,00
Credit account: Balance = -300,00
Withdrawal attempt failed.
```

interface TransferBankAccount

```
public interface TransferBankAccount extends BankAccount {
    boolean transferTo(BankAccount destination, double amount);
}

public class TransactionAccount extends DebitAccount
                        implements TransferBankAccount {
    public boolean transferTo(BankAccount destination, double amount) {
        boolean result;
        result = withdraw(amount);
        if (result) {
            destination.deposit(amount);
        }
        return result;
    }
    @Override
    public String toString() {
        return String.format("Transaction Account: Balance = %.2f",
                                                    getBalance());
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        BankAccount debit = new DebitAccount();
        debit.deposit(200);
        debit.withdraw(100);
        System.out.println(debit);
        debit.withdraw(200);

        BankAccount credit = new CreditAccount();
        credit.deposit(300);
        credit.withdraw(100);
        System.out.println(credit);
        credit.withdraw(500);
        System.out.println(credit);
        credit.withdraw(300);

        TransferBankAccount current = new TransactionAccount();
        current.deposit(500);
        System.out.println(current);
        current.transferTo(credit, 400);
        System.out.println(credit);
        System.out.println(current);
    }
}
```

```
Debit account: Balance = 100,00
Withdrawal attempt failed.
Credit account: Balance = 200,00
Credit account: Balance = -300,00
Withdrawal attempt failed.
Transaction Account: Balance = 500,00
Credit account: Balance = 100,00
Transaction Account: Balance = 100,00
```

Inconsistent interfaces

- When a class implements two interfaces:
  - Inconsistency can occur if several interfaces have constants with the same name but different values
  - Another inconsistency can occur if interfaces contain methods with the same name but a different return type
- If a class implements two inconsistent interfaces, the class definition is illegal

```
public interface Articulate {
    void makeSound();
}
abstract class Animal {
    protected String name;
    public Animal(String n) { name=n; }
}

class Dog extends Animal implements Articulate {
    Dog(String n) { super(n); }
    @Override
    public void makeSound() {
        System.out.println(name+ ": Bark bark");
    }
}
class Cat extends Animal implements Articulate {
    Cat(String n) { super(n); }
    @Override
    public void makeSound() {
        System.out.println(name+ ": Meow meow");
    }
}
```

`class Fish extends Animal …`
But does not implement Articulate

```
public class Main {
    public static void main(String[] args) {
            Articulate [] a = {
                    new Cat("Pussy"),
                    new Dog("Puppy")
            };

        for(Articulate x : a) {
            x.makeSound();
        }
    }
}
```

```
Pussy: Meow meow
Puppy: Bark bark
```

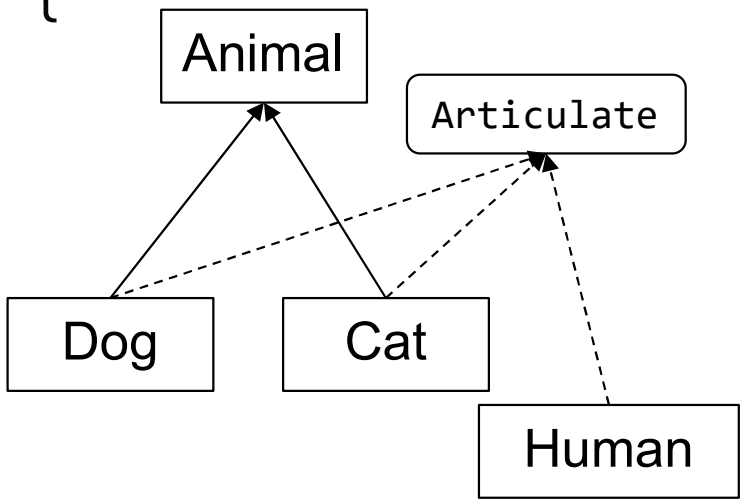
```
public class Main {
    public static void main(String[] args) {
            Articulate [] a = {
                    new Cat("Pussy"),
                    new Dog("Puppy")
            };

        for(Articulate x : a) {
            //x.makeSound();
            communicate(x);
        }
    }
    static void communicate(Articulate a) {
        a.makeSound();
    }
}
```

```
Pussy: Meow meow
Puppy: Bark bark
```

```
public class Human implements Articulate {
    protected String name, surname;
    public Human(String n, String s) {
        name=n;
        surname = s;
    }


    @Override
    public void makeSound() {
        System.out.println(name + " " + surname
                + ": I can speak");
    }
}
```



```
public class Main {
    public static void main(String[] args) {
            Articulate [] a = { new Cat("Pussy"),
                    new Dog("Puppy"),
                    new Human("John", "Smith")
            };

        for(Articulate x : a) {
            //x.makeSound();
            communicate(x);
        }
    }
    static void communicate(Articulate a) {
        a.makeSound();
    }
}
```

```
Pussy: Meow meow
Puppy: Bark bark
John Smith: I can speak
```

`Comparable` interface

- The **`Comparable`** interface is found in the **`java.lang`** package and is automatically available to any program
- It contains only one method that needs to be implemented:

```
public int compareTo(Object other);
```

`Comparable` interface

- The **`compareTo`** method must return:
  - A negative number if the calling object is the "before" the parameter
  - Zero, if the calling object is "equal to" the parameter
  - A positive number if the calling object is the "after" the parameter
  - If the **`other`** parameter is not of the same type as the class in which it is defined, then a **`ClassCastException`** exception should be thrown

`Serializable` interface

- An extreme example of an interface is the **`java.io.Serializable`** interface
  - It does not contain any methods and or defined constants: it is completely empty (a.k.a. marker interface)
  - It is used to indicate that the class implementing it supports serialization
  - Its purpose is to signal to the JVM that a class's instances are eligible for serialization.

`Cloneable` interface

- **`Cloneable`** is another unusual interface in Java
  - It also does not contain methods or constants (a.k.a. marker interface)
  - It is used to indicate how the **`clone`** method (inherited from the **`Object`** class) should be used and redefined
  - Its purpose is to signal to the JVM that a class's instances are eligible for cloning.

`Cloneable` interface

- If the data of the object being cloned includes instance variables of a mutable class, then the simple implementation of **`clone`** (shallow copy) will cause a *privacy leak*
- Cloneable merely gives permission for **`Object.clone()`** to be invoked without throwing an exception. However, it will perform only field-for-field copy (shallow copy) of instances of the class that implements it.
- By convention, classes that implement **`Cloneable`** should override the protected **`Object.clone()`** method with a public version that should handle the specific cloning logic for the class.
- When implementing the **`Cloneable`** interface in the body of the overridden **`clone()`** method:
  - First the **`clone`** method of the base class is invoked (**`super.clone()`**)
  - Then create independent copy of all variable instances of a mutable class (by calling their **`clone`** method)

Interfaces since Java 8

```
public interface Addressable
{
    String getStreet();
    String getCity();
    default String getFullAddress()
    {
        return getStreet() + " , " + getCity() ;
    }
}
```

- Predefined implementation of a method defined in the interface is allowed
- Any class that implements the interface can use the predefined implementation without the obligation to override it (but can provide its oven implementation overriding the provided default)

Interfaces since Java 8

```
class Letter implements Addressable {
    private String street;
    private String city;

    public Letter(String street, String city) {
        this.street = street;
        this.city = city;
    }
    @Override
    public String getCity() {
        return city;
    }
    @Override
    public String getStreet() {
        return street;
    }
    public static void main(String[] args) {
        // Test the Letter class.
        Letter l = new Letter("123 AnyStreet", "AnyCity");
        System.out.println(l.getFullAddress());
    }
}
```

Interfaces since Java 8

```
public interface X
{
    static void foo() {
        System.out.println("foo");
    }
}

class Y implements X
{
}

public class Main {
    public static void main(String[] args)
    {
        Y obj();
        X.foo(); // OK
        // Y.foo(); // won't compile
        // obj.foo(); // won't compile
    }
}
```

- Static methods defined in an interface are part of the interface and cannot be overridden in the classes that implement the interface
- They can only be invoked through the interface name, not through the class name or through objects of the class

Interfaces since Java 8

```
public interface Interface1 {
        void method1(String str);
        default void log(String str){
            System.out.println("I1 logging::"+str);
        }
    }
public interface Interface2 {
        void method2(String str);
        default void log(String str){
            System.out.println("I2 logging::"+str);
        }
    }
```

If a class is implementing both interfaces, it will have to provide implementation for log() method otherwise compiler will throw compile time error.

```
public class MyClass implements Interface1, Interface2 {
    @Override
    public void method2(String str) {  }
    @Override
    public void method1(String str) {  }
    @Override
    public void log(String str){
        System.out.println("MyClass logging::"+str);
        //Interface1.log("abc");
    }
    public static void main(String[] args) {
        MyClass myClass = new MyClass();
        myClass.method1("xyz");
        myClass.method2("xyz");
        myClass.log("abc");
    }
}
```

Interfaces past Java 8

- **Private methods** in Java interfaces were introduced in Java 9 to provide a way to encapsulate helper code that is not part of the public API of the interface
- **Sealed Interfaces** were introduced in Java 16 to give authors strict control over who can implement their contract.

```
// Only Circle and Square are allowed to implement Shape
public sealed interface Shape permits Circle, Square {
double calculateArea(); }
public final class Circle implements Shape { ... }
public final class Square implements Shape { ... }
```


Questions?